BMIT2073

Mobile Application Development

Version 202605

Contents

Introduction

Welcome to BMIT2073 Mobile Application Development course this semester. Get ready to embark on an existing and practical journey into the world of creating mobile applications. This set of practical sessions are created using the following tools:

- Android Studio
- Android SDK Platform
- Android SDK Build Tools
- Flutter
- Dart

The official Flutter website is your primary and most reliable source for learning Flutter and finding help.

Learn Flutter (docs.flutter.dev/get-started/learn-flutter): This section is specifically designed for new Flutter developers and provides a structured path to learn the fundamentals.

- Dart language overview: Since Flutter uses Dart, it's recommended to have a basic understanding of the language. The documentation offers an overview for those with experience in other object-oriented languages.
- Write your first Flutter app (Codelab): This interactive tutorial walks you through creating a simple Flutter application for mobile, desktop, and web.
- Learn the fundamentals: This section guides you through the most important aspects of building Flutter applications.

Flutter Cookbook (docs.flutter.dev/cookbook): This is a fantastic resource with "recipes" that demonstrate how to solve common problems in Flutter apps. It covers various topics like:

- Animation
- Design (e.g., adding drawers, snackbars, custom fonts, themes)
- Forms and Gestures (e.g., form validation, handling taps)
- Navigation & Routing
- Accessing device APIs (e.g., camera, gallery)
- Networking and Data Persistence
- Testing (unit, widget, integration)

Widget Catalog: Flutter's UI is built with widgets. The documentation has a comprehensive catalog of all available widgets, their properties, and examples of how to use them.

FAQ (docs.flutter.dev/resources/faq): Provides answers to frequently asked questions about Flutter, its capabilities, technology, and more.

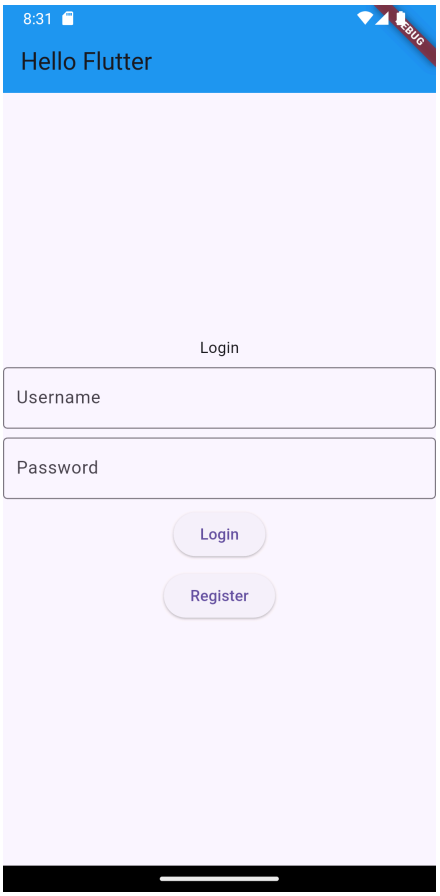
Samples (github.com/flutter/samples): The official GitHub repository for Flutter contains a wide range of example applications and demos that showcase various Flutter features and best practices. This is invaluable for seeing real-world implementations.

A crucial aspect of this course is our practical lab sessions. We will provide you with code snippets and examples to help you understand core concepts. We strongly urge you to focus on observing and understanding the functionality of the code provided, rather than simply copying and pasting.

Furthermore, **towards the end of each lab session, you will find a "TODO" item.** These are tasks designed for you to complete independently, applying the concepts learned during the lab.

Practical 1: Setup Flutter Development Environment

Flutter is Google's UI toolkit for building applications for mobile, web, and desktop from a single codebase. In this practical, you will install the necessary files and build the following Flutter application:



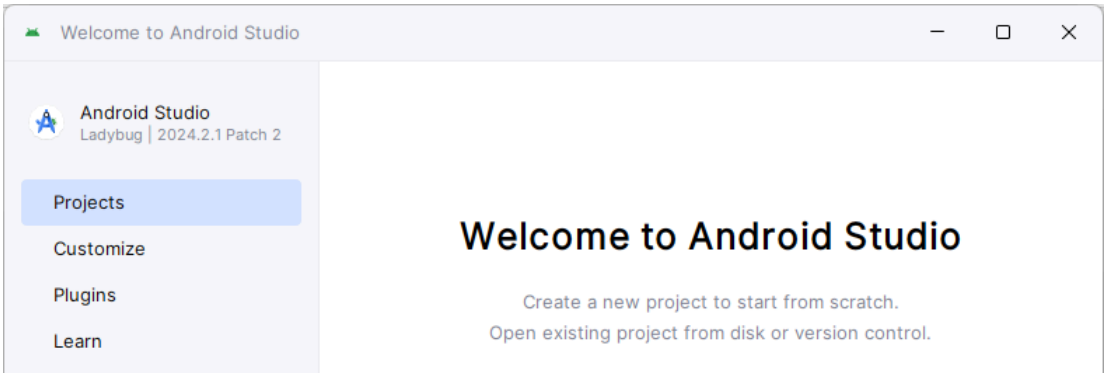
1. Download the following installation files:
    a. Flutter SDK (https://docs.flutter.dev/get-started/install)
    b. Android Studio (https://developer.android.com/studio)
    c. Git [Optional] (https://git-scm.com/downloads)
    d. Node.Js [Optional] (https://nodejs.org/en/download/current)

2. Extra the Flutter SDK to a local drive. E.g. C:\flutter\

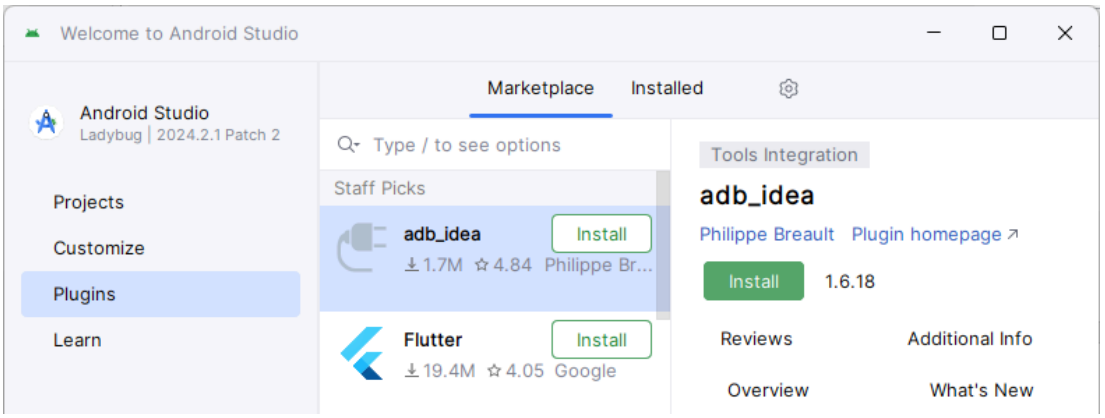
3. Install Android Studio.

4. Launch Android Studio and install the following components:
    a. Android SDK Platform
    b. Android SDK Command-line Tools
    c. Android SDK Build-Tools
    d. Android SDK Platform-Tools
    e. Android Emulator

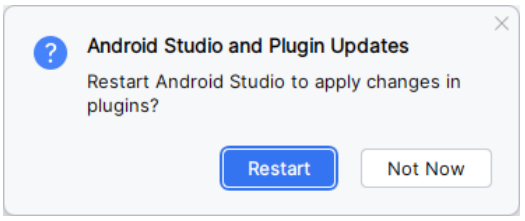
5. Upon completion of Android Studio installation, on the welcome screen, click **Plugins**.



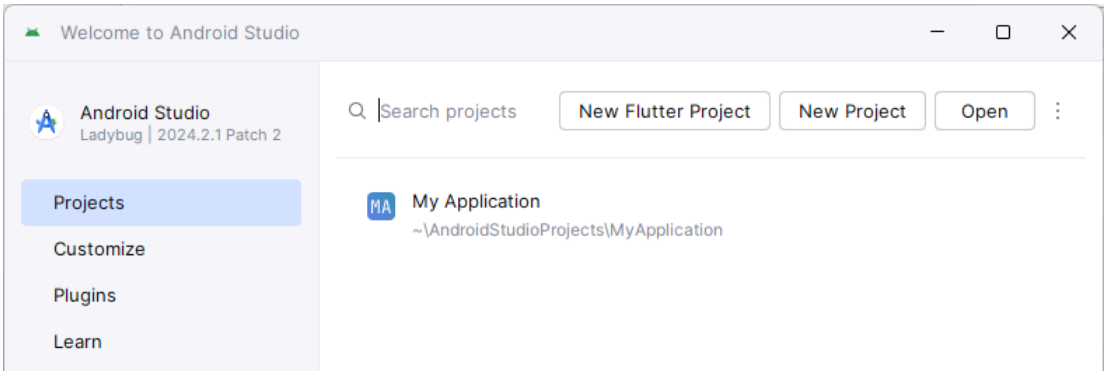
6. Under the Marketplace tab, search the keyword **flutter**.



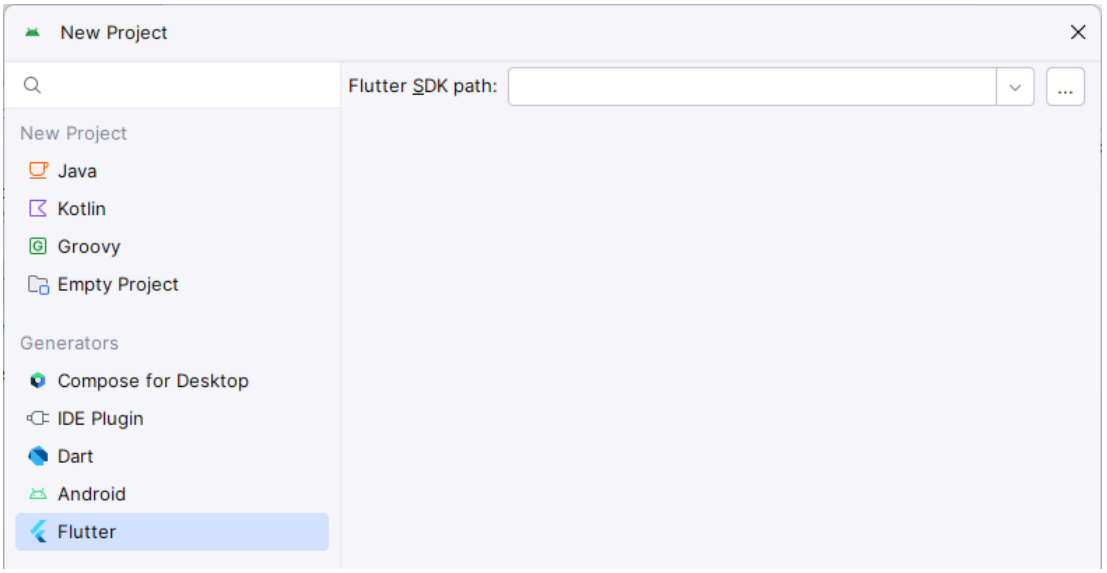
7. Click the Install button next to Flutter. Once the installation is completed, press the Restart IDE and "Restart".



8. Next, on the Welcome Screen, click **New Flutter Project**.



9. Click **Flutter** on the left panel. Enter the **Flutter SDK path** (E.g. C:\flutter\) and click **Next** button.



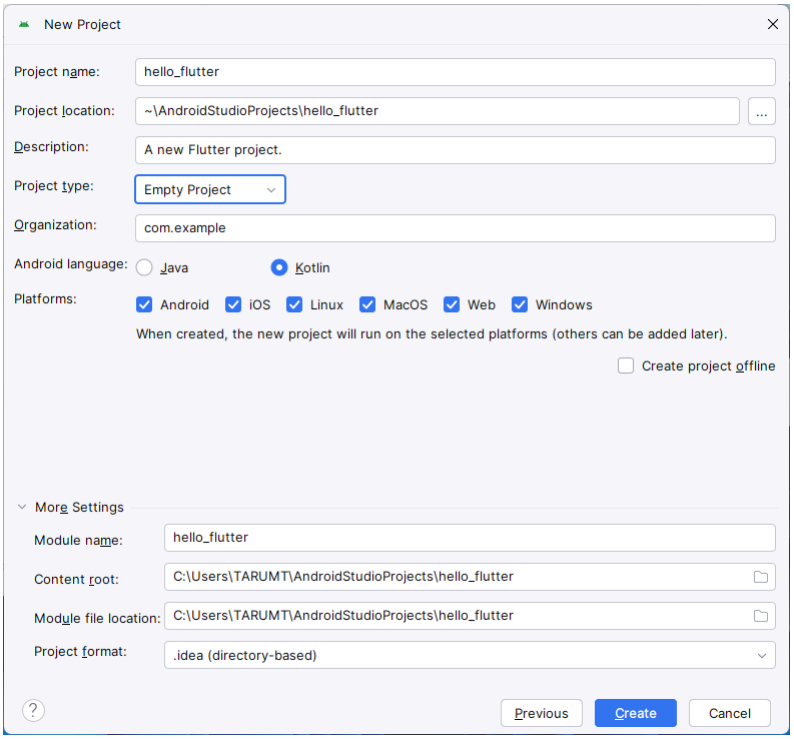
10. Next, you will be prompted to enter a project name and select other settings. Choosing a good flutter project name:

Field	Description
Project name	Use lowercase letters and underscores. E.g. my_todo_app.
Project location	A dedicated folder for your Flutter project.
Description	A readme file.
Project type	Determine the kind of application you are building.
Organization	Use your company's domain as a prefix for the project name. Example: com.yourcompany.myapp This helps to organize projects and avoid naming conflicts.
Android language	Kotlin and Java
Platforms	Select the platforms you want to support (Android, iOS, Web, Desktop).
Module name	Same as the project name.

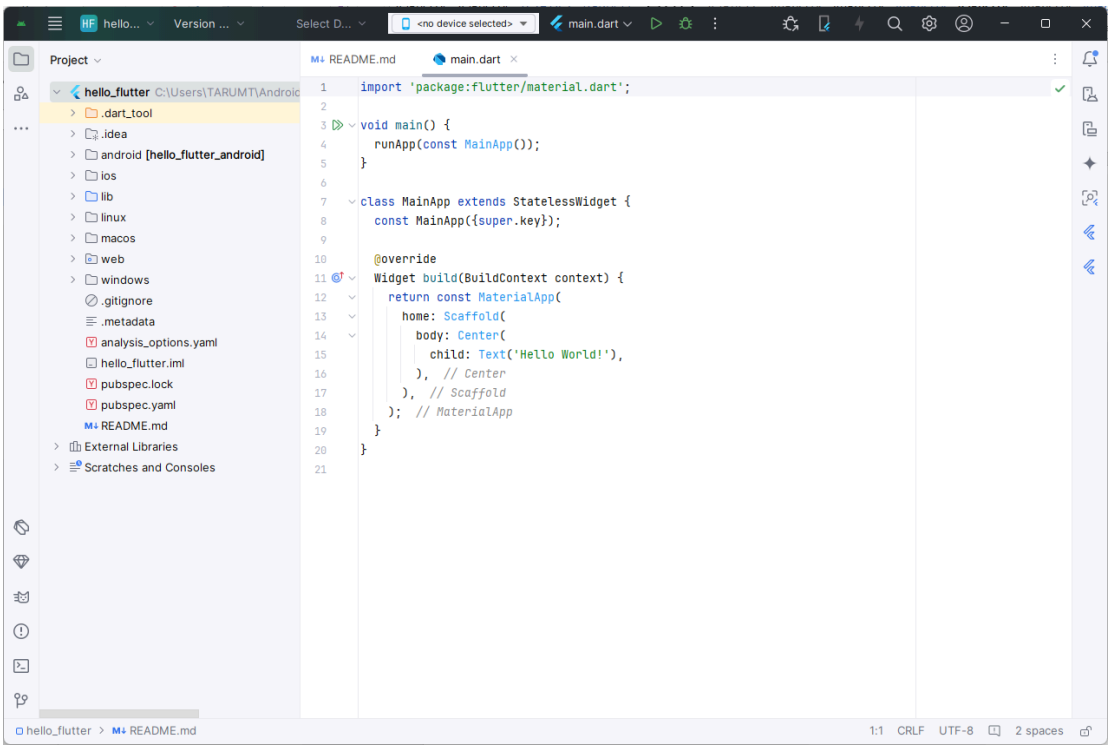
Type	Description
Application	Creates a standalone Flutter application that can be deployed to various platforms like Android, iOS, Web, Windows, and macOS.
Plugin	To provide access to platform-specific APIs (e.g., camera, sensors, Bluetooth) from within your Flutter code.

Package	To create reusable code modules that can be shared across multiple Flutter projects or published to pub.dev (Flutter's package repository).
Module	Creates a reusable Flutter module that can be integrated as a dependency into other Flutter projects.
Skeleton	A simplified project with minimal boilerplate code.
FFI Plugin	To interact with native libraries (C, C++, etc.) using the Foreign Function Interface (FFI) in Dart.
Empty Project	Creates a completely empty project with no default files or structure.

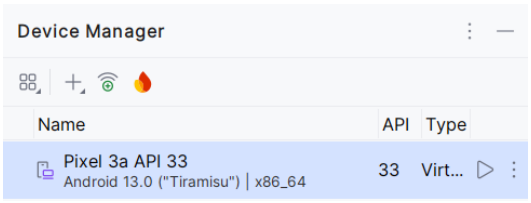
11. For this practical we select an **Empty Project** type.



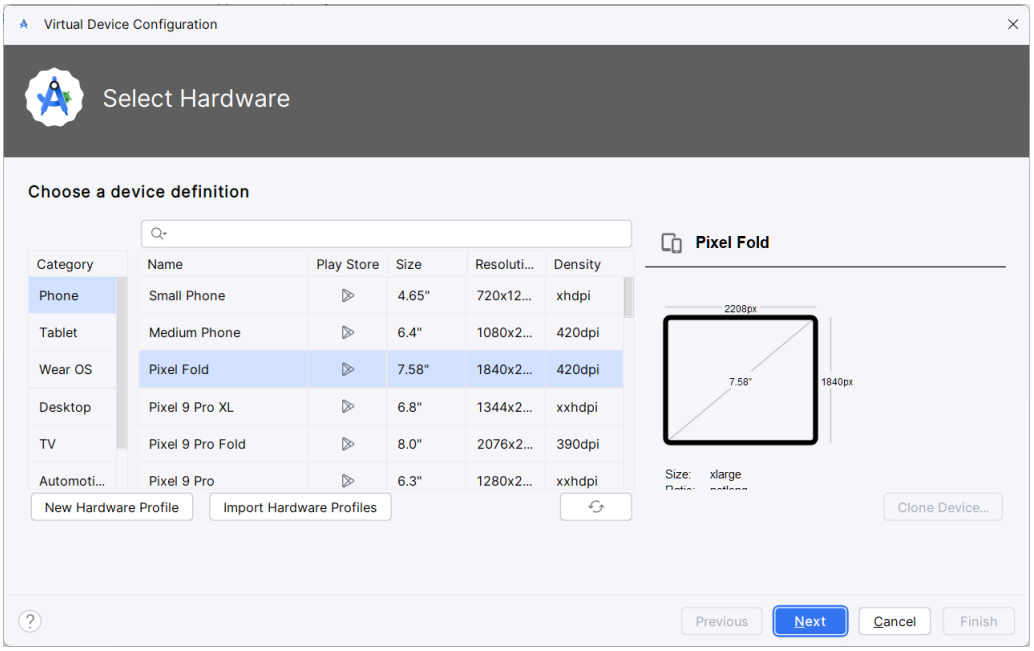
12. Once the project is ready you will see the screen as follows: -



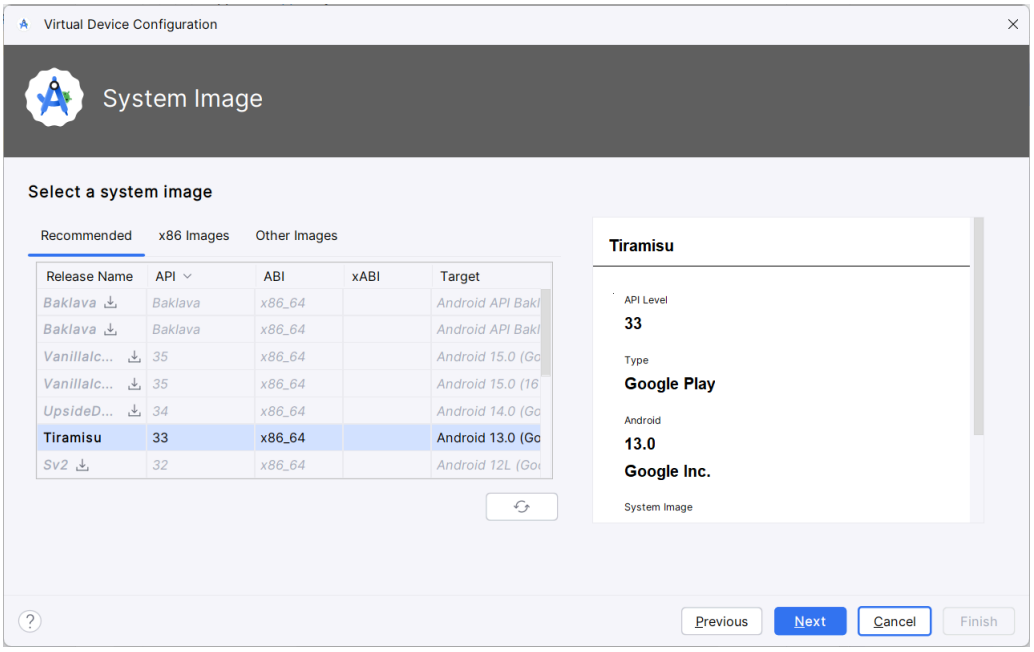
13. Click the **Device Manager** ( ) on the right panel and press the **Run** ( ) button.



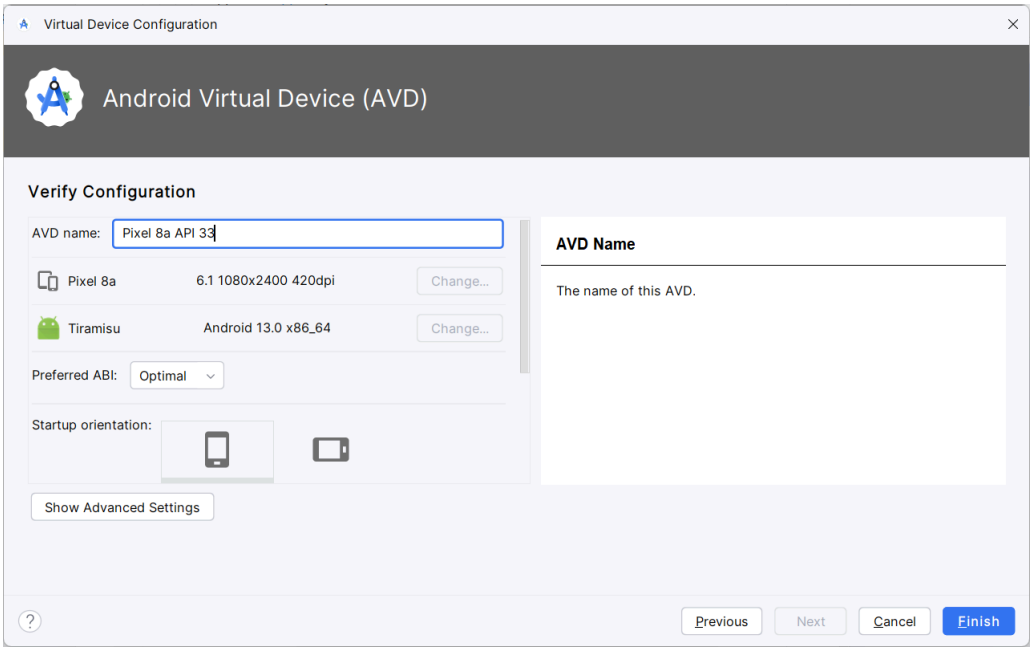
However, If you do not see any device in the **Device Manager**, you may click the **Add a new device** ( ) button and click **Create Virtual Device**. Select a device definition from the list and click the **Next** button.



Then, select a system image and click the Next button.

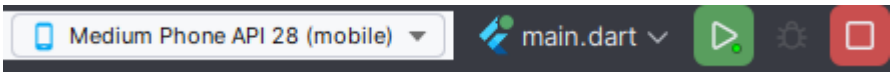


Enter a name in the **AVD** (Android Virtual Device) and click the **Finish** button.

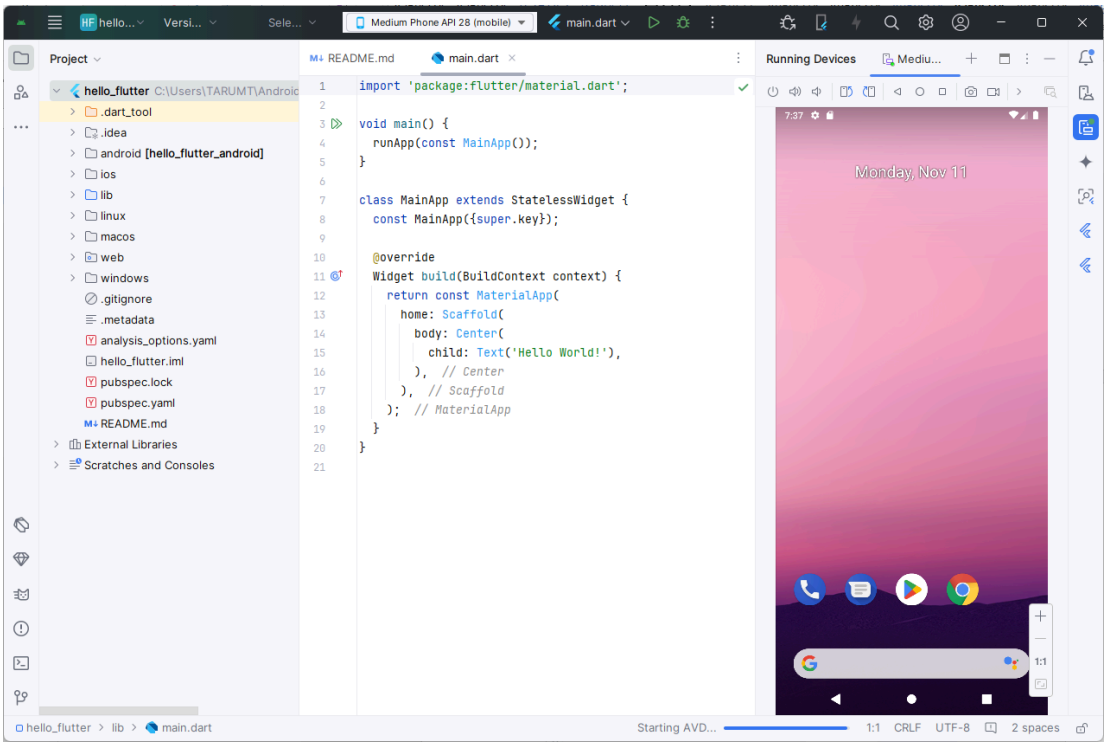


The newly created AVD should appear in the Device Manager.

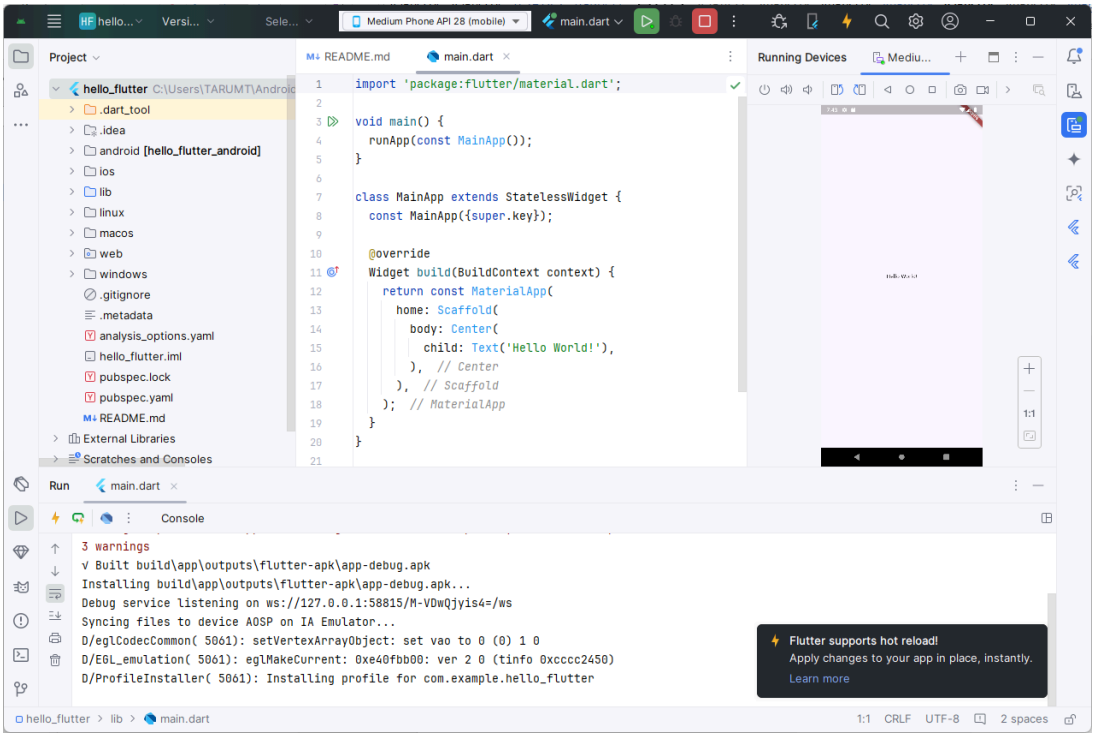
14. On the top of the screen, press the **Run** ( ) button. Running the app for the first time will take a long time depending on the quality of the network.



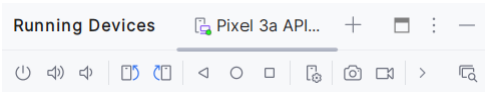
15. You will see the running device as follows: -



16. The app run as follows: -



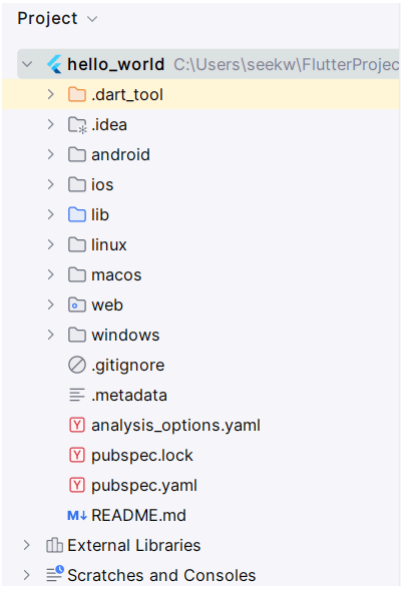
In the **Running Devices** tab ( ), examine the top panel that has a list of common functions associated with a mobile device such as On/Off button, volume up/down, etc.



17. At the bottom left of the screen, click the **Run** tab ( ), which shows the console messages. When you build your app (compile it into an APK or bundle), the console shows the progress of the build process, any warnings or errors encountered, and the final build results.

This panel also shows the logcat output. This is crucial for debugging. It displays system messages, application logs, and any errors or exceptions thrown by your app while running.

18. The project File Explorer is on the left. It lists all the platforms supported by your project.

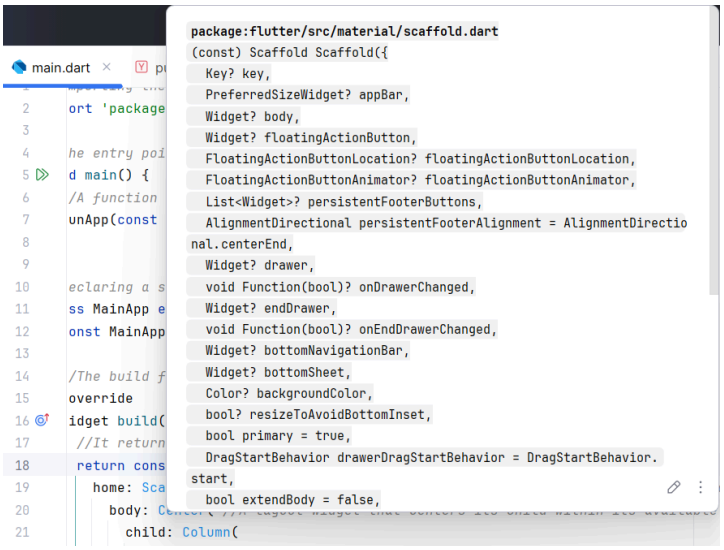


19. Expand the **lib** folder. This folder contains Dart program files for your project. Open and examine the **main.dart** file and enter the following comments so that you understand the purposes of these commands:

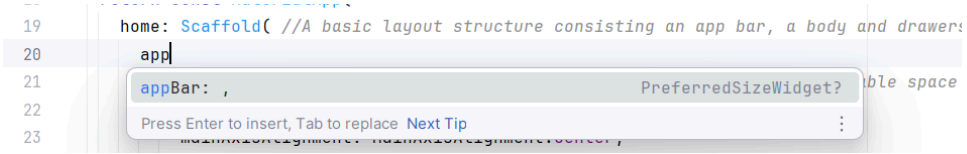
```dart
//Importing the necessary packages
import 'package:flutter/material.dart';

//The entry point of the Flutter application.
void main() {
  //A function that takes a Widget as an argument and renders it on the screen.
  runApp(const MainApp());
}

//Declaring a stateless widget class that does not change its state over time
class MainApp extends StatelessWidget {
  const MainApp({super.key}); //Constructor of this widget

  //The build function is responsible for building the widget's UI
  @override
  Widget build(BuildContext context) {
    //It returns a MaterialApp widget
    return const MaterialApp(
      //A basic layout structure consisting an app bar, a body and drawers
      home: Scaffold(
        //A layout widget that centres its child within its available space
        body: Center(
           child: Text('Hello World!'), //A text widget
        ),
      ),
    );
  }
}
```

Next, let us edit the program code further to change the UI.

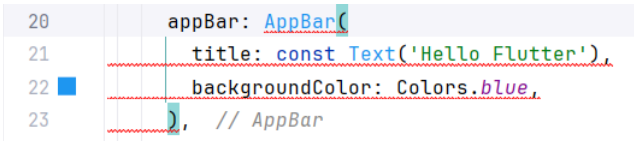
20. Mouse over the **Scaffold** widget to observe its attributes. You may scroll down to see all attributes and a short description about the widget.



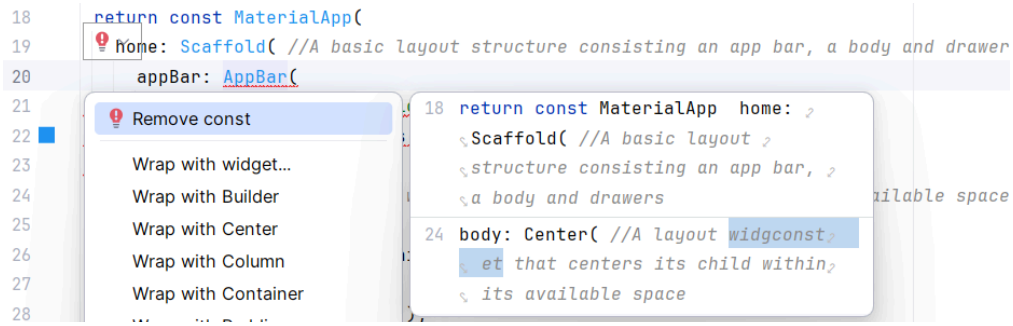
21. Within the **Scaffold** widget, enter the text **app**, the editor will find and show a matching attribute, press the **Tab** key to accept the suggested attribute.



22. Enter the following code (ignore the error at the moment):



23. If there is an error in your code, the editor might have solutions to fix it. Click any part of the error code and then click the **red light bulb** (💡) to see the possible fixes. Select **Remove const** as a fix.



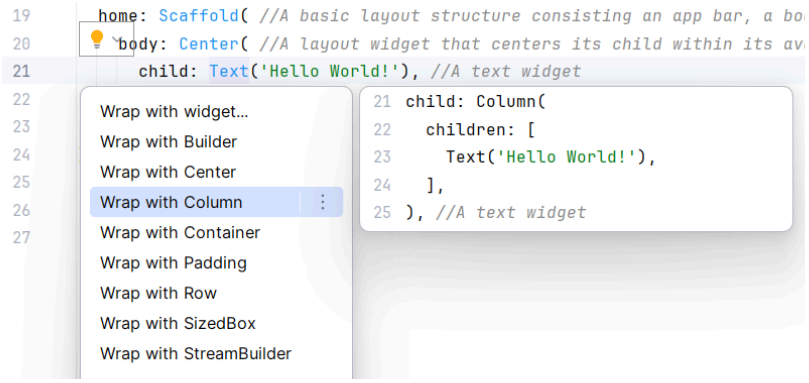
As suggested by the editor, the command **const** (it means constant) shall be removed from the **MaterialApp** widget after inserting the AppBar. The error arises because the **AppBar** widget within the **Scaffold** is not a constant constructor.

After applying the fix your code shall look like this:

```
return MaterialApp(
  home: Scaffold( //A basic layout structur
    appBar: AppBar(
      title: const Text('Hello Flutter'),
      backgroundColor: Colors.blue,
    ),  // AppBar
```

24. Save the program code (Ctrl+S) and observe the app bar. Flutter supports hot reload; therefore, you should see the UI changes without the need to relaunch your app.

25. At the moment, the **Center** widget can only hold one child widget. We need another widget that can hold multiple children. Mouse click on the **Text** widget, then click on the **yellow light bulb** (💡) on the right. You will see a drop-down list, select **Wrap with Column**. You can preview each selection on the right of each option.

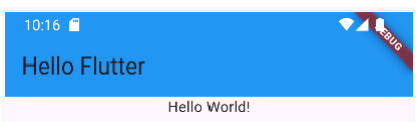


26. The result will be as follows: -

```
body: Center( //A layout widget that centers its
  child: Column(
    children: [
      Text('Hello World!'),
    ],
  ), //A text widget  // Column
),  // Center
```

Column is a widget that can contain multiple widgets within the array attribute named **children**.

27. Save your work and view the UI. Now, the **Text** widget is the only child under the **Column** widget.



28. Modify the **Column** widget so that all the children widgets are aligned to the centre of the screen as follows: -

```
child: Column(
  //A text widget
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text('Hello World!'),
  ],
), //A text widget
```

Save your work and observe the result.

29. Modify the existing Text widget. Also insert two TextField widgets to allow users to enter text.

```
Text('Login'),
TextField(
  keyboardType: TextInputType.name,
  decoration: const InputDecoration(
    labelText: 'Username',
    border: OutlineInputBorder(),
  ),
),
TextField(
  keyboardType: TextInputType.text,
  obscureText: true,
  decoration: const InputDecoration(
    labelText: 'Password',
    border: OutlineInputBorder(),
  ),
),
```

30. Insert another two ElevatedButton widgets:

```
ElevatedButton(
    onPressed: (){}, //Empty event handler
    child: Text('Login'),
),
ElevatedButton(
    onPressed: (){}, //Empty event handler
    child: Text('Register'),
),
```

Congratulations! You have completed your first Flutter app!

Remarks:

If you are unable to deploy a Flutter app in Android, do check the Gradle compatibility matrix at https://docs.gradle.org/current/userguide/compatibility.html

For Kotlin version 1.8, the minimum Gradle version is 8.0 with JVM between 8 and 23. JVM 24 and later versions are not yet supported.

TODO: Visit https://dart.dev/language and learn the basics of Dart language!

Practical 2: Setting up Version Control System

Version control, also known as source control, is the practice of tracking and managing changes to software code. Version control systems are software tools that help software teams manage changes to source code over time.

Android Studio supports a variety of version control systems (VCSs), including Git, GitHub, CVS, Mercurial, Subversion, and Google Cloud Source Repositories.

1. Download and install Git. (https://git-scm.com/downloads)

2. Open the **Git Command** or **Windows Command-prompt** or **Android Studio command prompt**, enter the following commands:

```
git config --global user.name "Your GitHub ID"
git config --global user.email "Your Email Address"
```

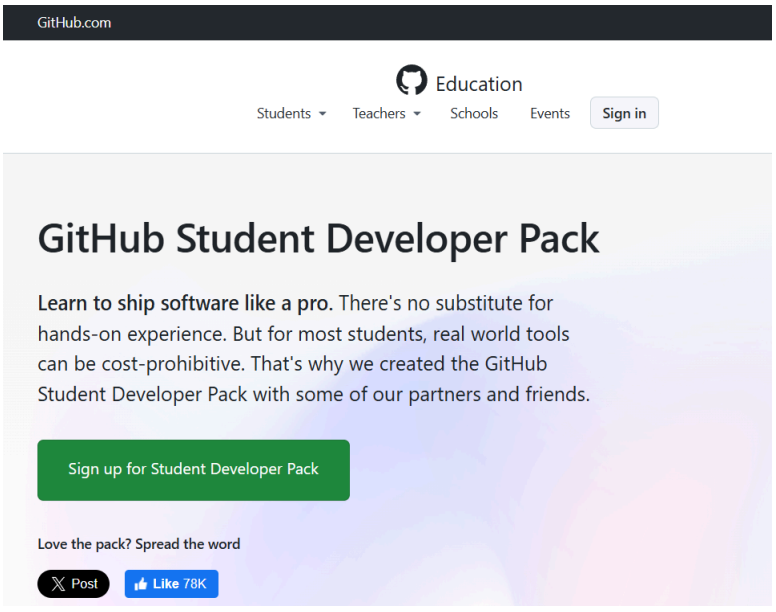
Confirm that you have set the Git username correctly, enter the command:

```
git config --global user.name
git config --global user.email
```

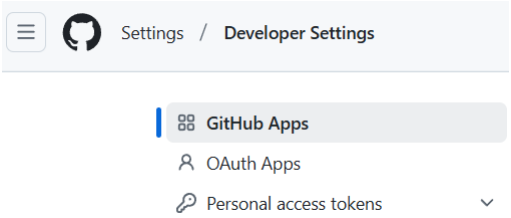
And you shall see your name:

```
> Your name here
> Your email here
```

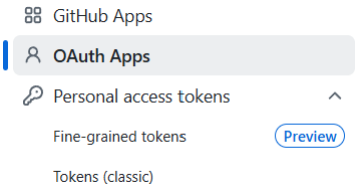
3. Register a new GitHub student account. (https://education.github.com/pack)



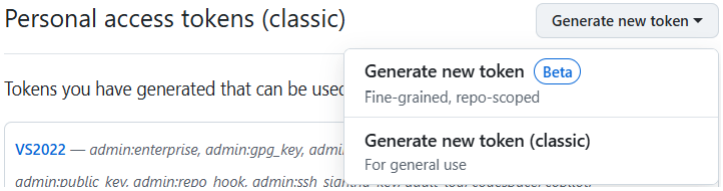
4. Login to your GitHub account. Click on your **profile picture** on the top-right corner, and click **Settings** (Settings).
5. On the left panel, click the last item named **Developer settings** (Developer settings). You shall see the Developer Settings list as below:



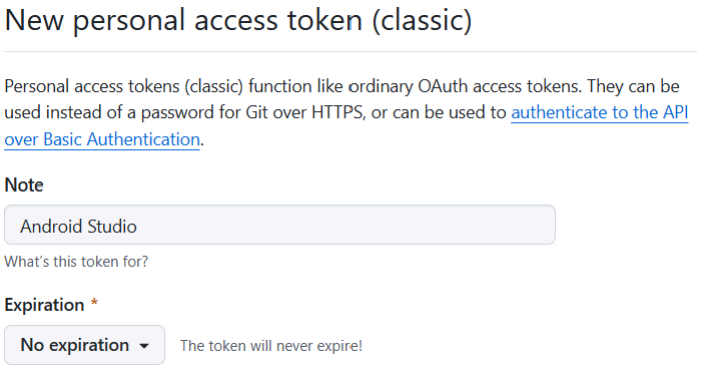
6. Click on the **Personal access tokens**, and select **Tokens (classic)**. Personal access tokens are an alternative to using passwords for authentication to GitHub when using the GitHub API, the command line, or an IDE.



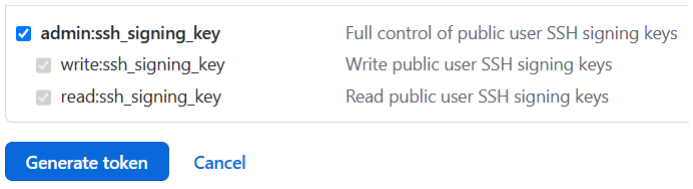
7. On the right of the **Personal access tokens (classic)** panel, click the **Generate new token** button, and select **Generate new token (classic)**.



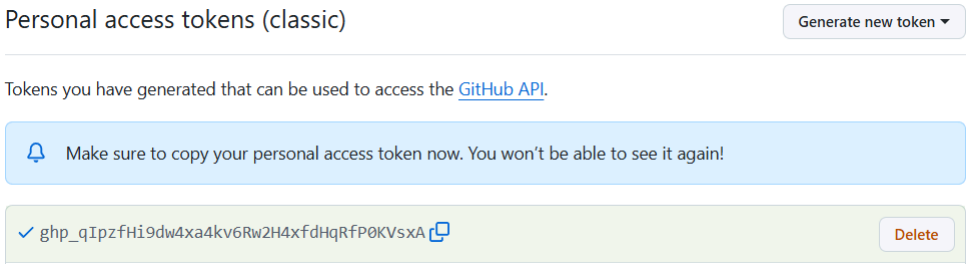
8. Assign a **Note** (e.g. Android Studio) to the token and set the **Expiration** to **No expiration**.



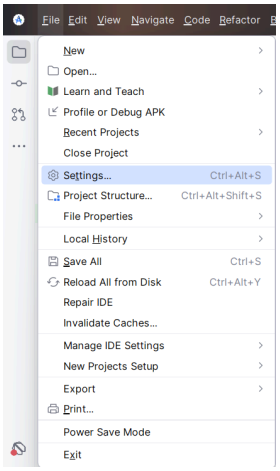
9. Select scopes associated with the token and click the **Generate token** button.



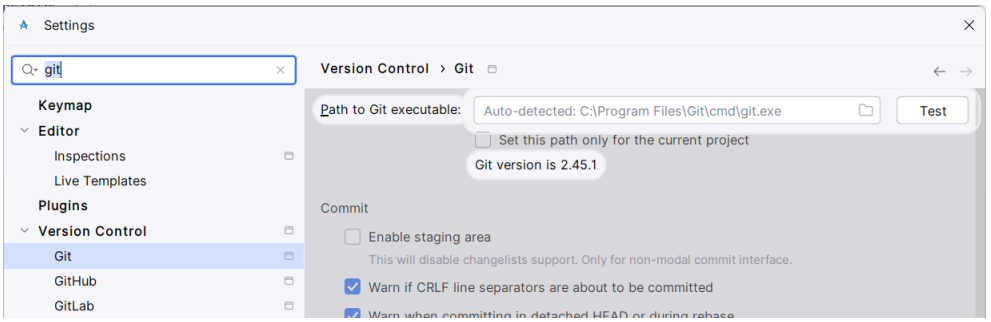
10. Click the **Copy** ( ) button once the token has been generated.



11. Next, open a new Flutter project file. Click on the **File** menu and **Settings**.

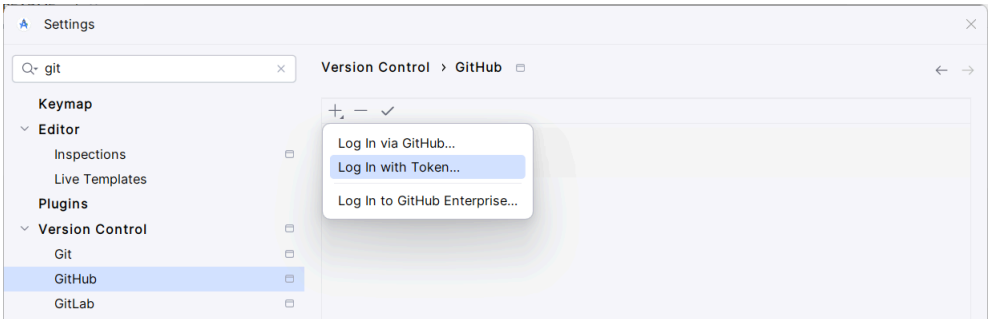


12. In the search bar, enter 'git'. Under the heading **Version Control**, click **Git**.

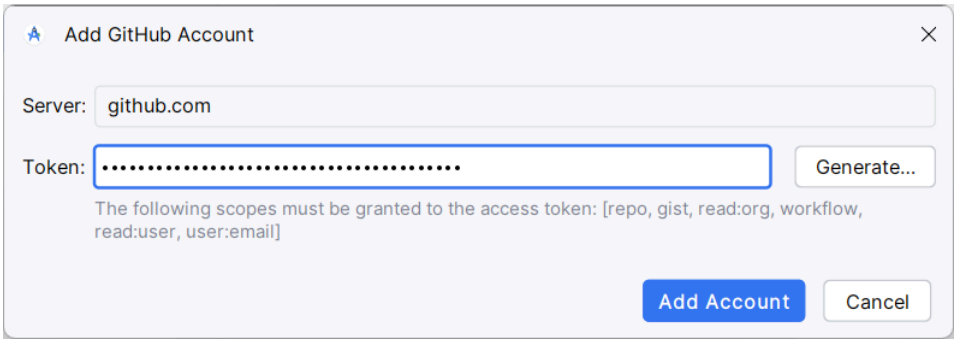


In the **Path to Git executable**, assign the git.exe path and press the **Test** button. You shall see the version code appear below the path.

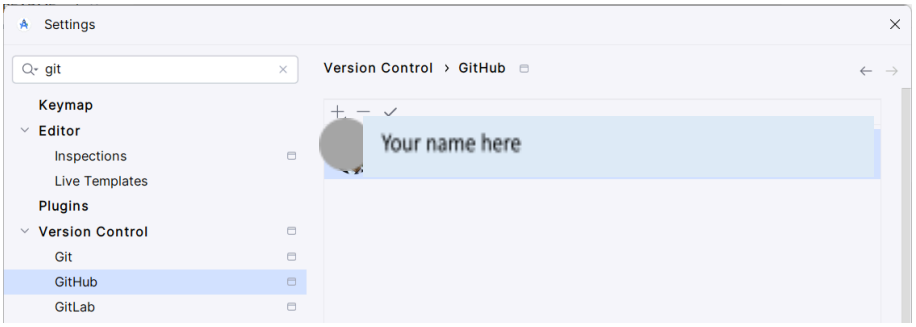
13. Next, click **GitHub** and press the add ( ) button. Click the **Login with Token…**



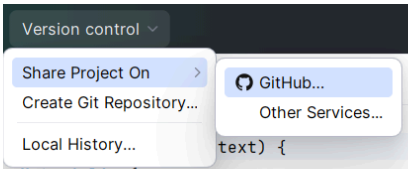
14. Paste the newly token to the Add GitHub Account screen and click the **Add Account** button.



15. Once completed, you should be able to see your profile appear in the list. Close the Settings panel.

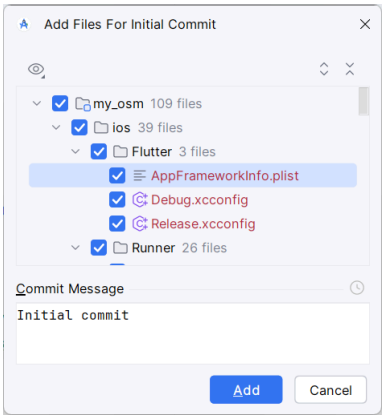


16. On the menu, click **Version Control** and select **Share Project On** -> **GitHub**…

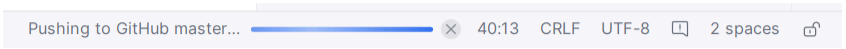


17. When prompted, enter a repository name, set the visibility (Private or Public), and also enter a short description about the project. Click the **Share** button to continue.

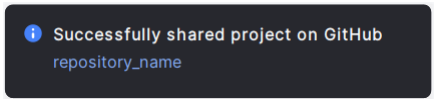
18. You will be prompted to add files for the initial commit. Review files to be added to the repository and press the **Add** button to continue.



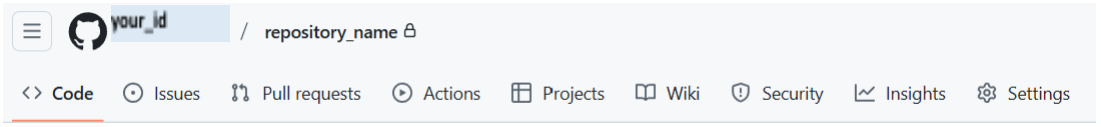
19. At the bottom right of the screen, you can observe the progress of pushing the files to the GitHub cloud.



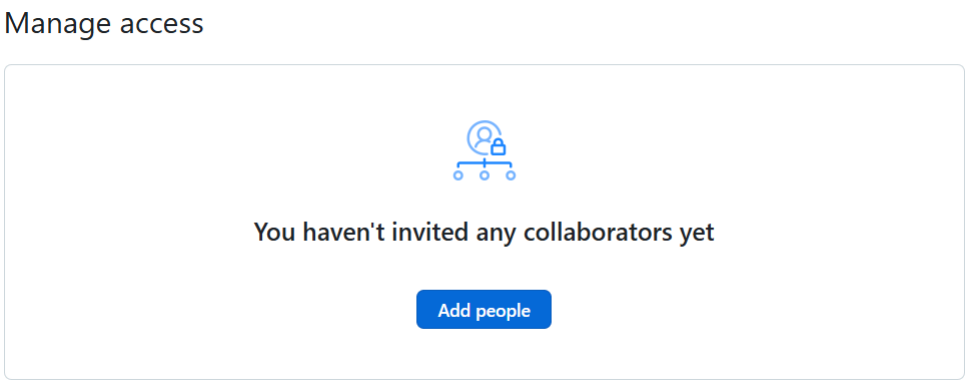
20. If everything goes well, you will see a message indicating the project has been shared on GitHub successfully. Click the given link to view the project files.



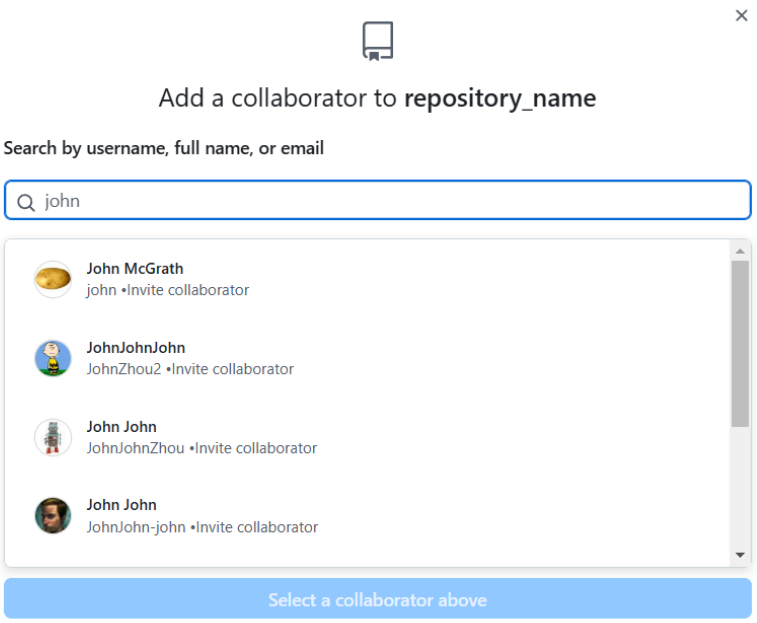
21. Your project will be shown in a browser. You may invite your team members to view, edit and update the project. Click the Settings (Settings).



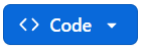
22. Select Collaborators (Collaborators). Under the heading **Manage access**, click the **Add people** (Add people) button.



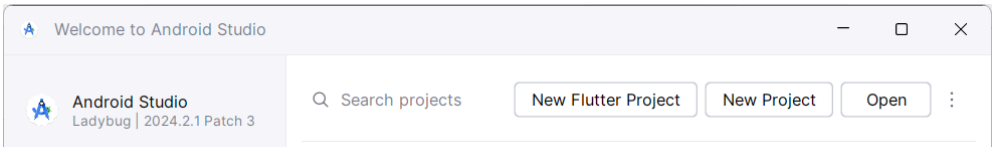
Enter IDs of your team member in the space provided:



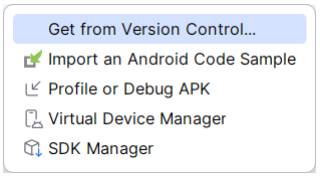
Click Select a collaborator from the list and click **Add [Team Member Name] to this repository**.

23. You may also share your repository to a member who would like to view your repository, however, not able to edit or update the repository. Click the **<> Code** tab and then the **Code** () button to show the link (with extension .git) for the project. Click the **Copy** ( ) button next to the clone link. You may share the git link to a member who would like to clone a copy of your project.

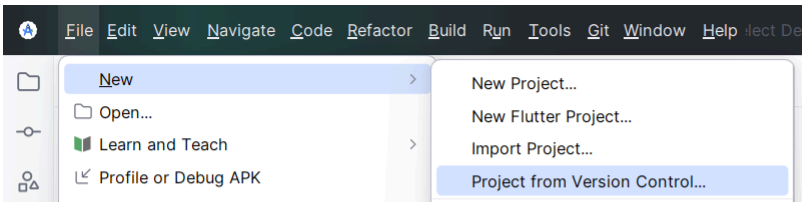
24. To clone a copy of the project, open Android Studio. Click on the More Actions (⋮) button.



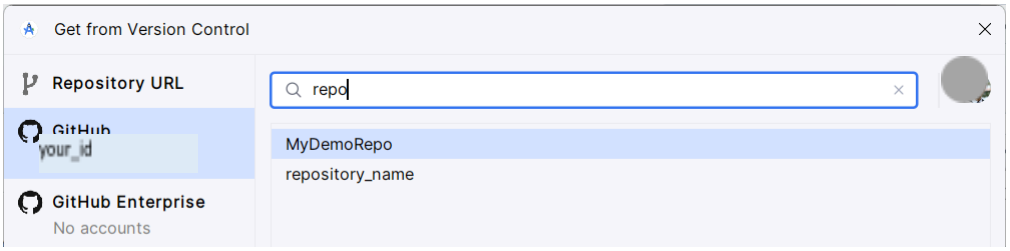
Select **Get from Version Control…**



Or, if you have opened an existing project, click **File** -> **New** -> **Project From Version Control…**

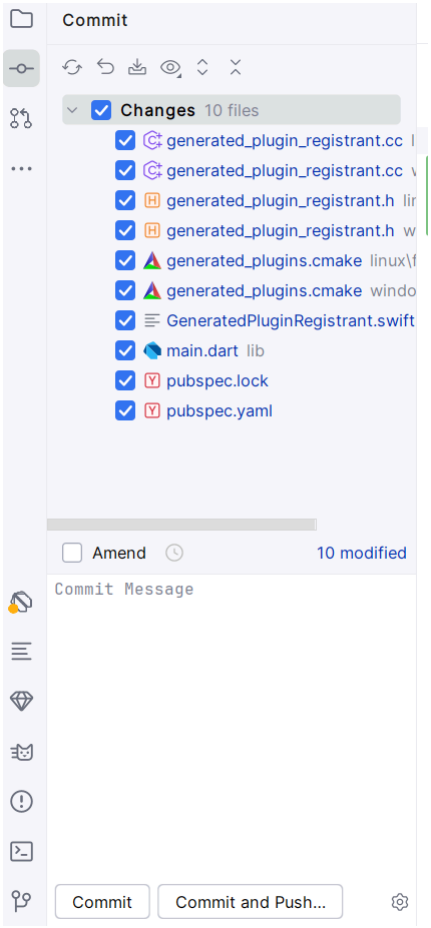


25. On the left panel, select your GitHub account and enter a search key (E.g. repo). Your project will appear in the repository list. Press the Clone (Clone) button to proceed.



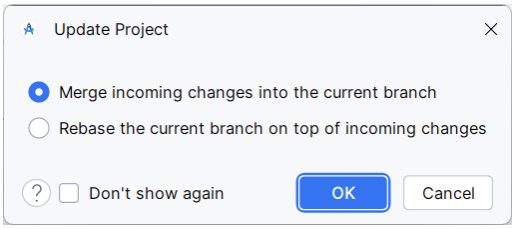
That's all you need to create and clone a repository.

26. After cloning a project, you may make some changes to the project files/codes/etc. Once you have completed all the necessary changes, you must **commit** the project. Click the VCS (main) button and then Commit (Commit...) button. Check all the files in the Commit panel.



Enter a commit message and click the **Commit and Push** button.

To view changes made by other team members, on the VCS menu, click the Update Project (Update Project...) button. The system will prompt you to Update the project:



TODO: Visit https://docs.github.com/en/get-started/start-your-journey/hello-world and learn GitHub essentials!

Practical 3: Inputs and Outputs

This lab focuses on getting inputs from the user, performing some computations and displaying the outputs. We will build a simple mobile app to calculate the Body Mass Index (BMI). It is a handy tool for measuring overweight and obesity. It can be calculated with your weight and height. In a gist, BMI is a good indicator for diagnosing body fat percentage and potential diseases.

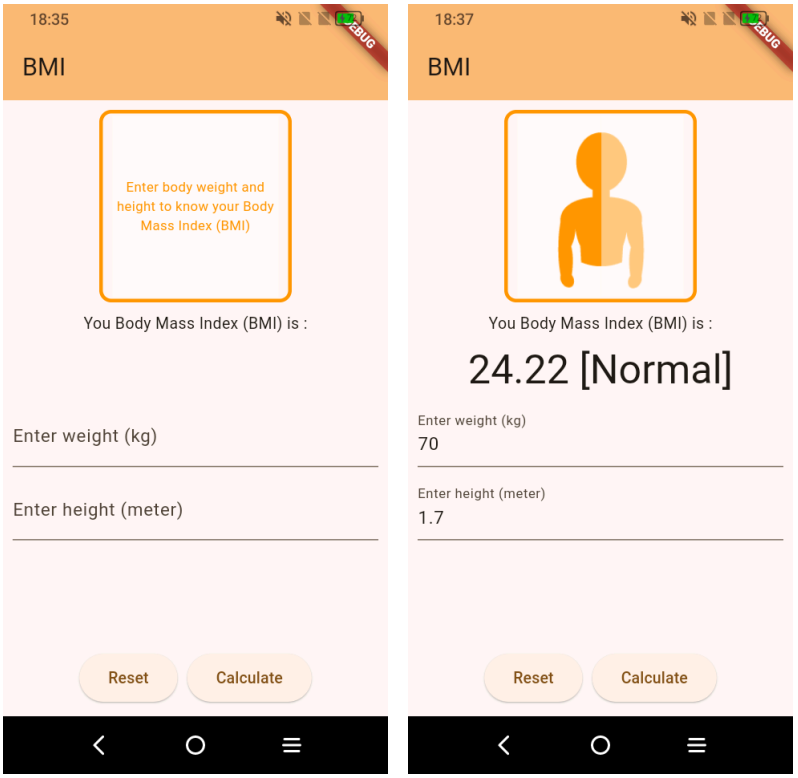
The formula for BMI calculation is as follows: -

$$BMI = weight / (height\ in\ meters)^2$$

BMI values and types are as follows: -

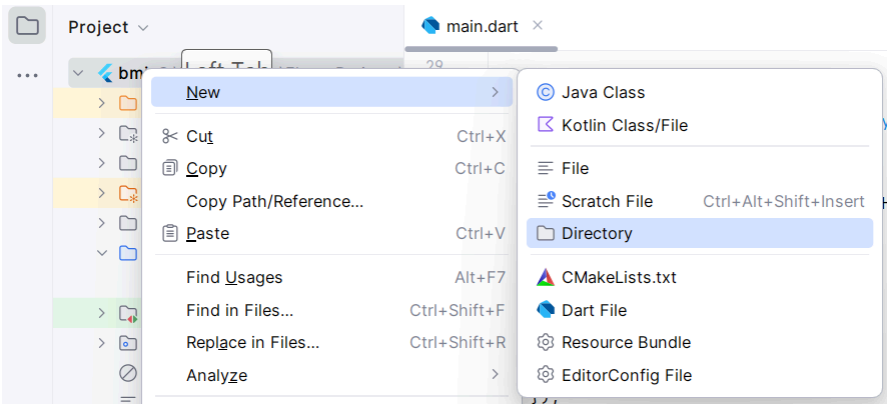
BMI	Value
Underweight	Below 18.5
Normal	18.5 – 24.9
Overweight	More than 25

Sample outputs of the app are as follows: -

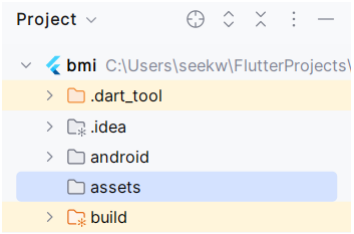


1. Create a new **Application** project. Set the project title to **bmi**.

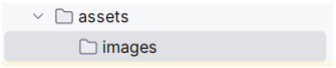
2. In the project **File Explorer**, right click on the **root** of the folder and select **New** -> **Directory**.



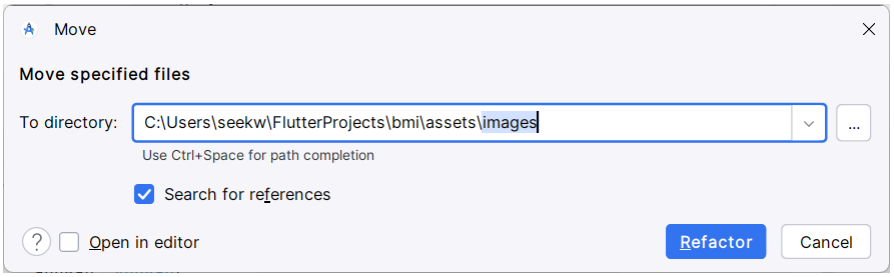
3. Name the new directory **assets**. The assets directory is dedicated for you to store static resources (images, audios, videos, fonts, JSONs, etc) that your app needs to function. These resources are then bundled with your app during the build process and become accessible at runtime.



4. Repeat step 2, within the assets directory, create a new sub-directory named **images**. We will place several images into this folder.

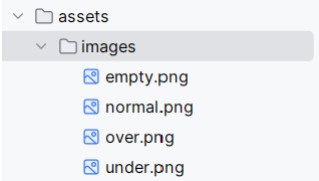


5. Copy and paste the four image files that represent BMI status into the **assets/images/** directory. When you paste the image files into the project File Explorer, the editor will prompt the following: -



Click the **Refactor** button to continue.

6. After that, the four image files should appear in the directory.



7. In the project File Explorer, open the **pubspec.yaml** file. Then scroll to the **flutter:** section. You are to insert a sub-section named **assets:**

```
flutter:

  # The following line ensures that the Material Icons font is
  # included with your application, so that you can use the icons in
  # the material Icons class.
  uses-material-design: true

  #TODO 1 : Insert assets
  assets:
    - assets/images/
  # To add assets to your application, add an assets section, like this:
  # assets:
  #   - images/a_dot_burr.jpeg
  #   - images/a_dot_ham.jpeg
```

Below the **assets:** section, insert **– assets/images/**. This ensures the app can bundle all the images in this directory.

8. Open the **main.dart** file, at the beginning of the file, insert the following code:

```
import 'dart:math';
```

The import statement is to make code from external libraries accessible within your current file. In this case, we will use the power function (**pow**). Usually, the editor will insert unambiguous libraries automatically.

9. Within the class **_MyHomePageState**, we declare some local variables and TextEditingControllers.

```
//TODO 2: Declare local variable
double _bmi = 0.0;
double _weight = 0.0;
double _height = 0.0;

String _bmiOutput='';
String _bmiImage = 'assets/images/empty.png';

final TextEditingController _weightCtrl = TextEditingController();
final TextEditingController _heightCtrl = TextEditingController();
```

The **TextEditingController** is a class that manages the text input within a **TextField** widget. It can get from and set text to a TextField.

10. Insert the **_calculateBMI()** function to get user inputs and calculate the value of BMI.

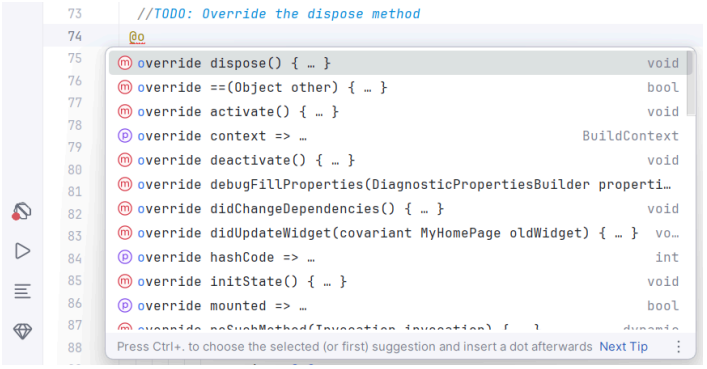
```
//TODO 3: Calculate BMI value
void _calculateBMI() {

  _weight = double.tryParse(_weightCtrl.text)!;
  _height = double.tryParse(_heightCtrl.text)!;

  setState(() {
    _bmi = _weight / pow(_height, 2);

    if (_bmi < 18.5) {
      _bmiImage = 'assets/images/under.png';
      _bmiOutput = '${_bmi.toStringAsFixed(2)} [Underweight]';
    }else if(_bmi >=25 ){
      _bmiImage = 'assets/images/over.png';
      _bmiOutput = '${_bmi.toStringAsFixed(2)} [Overweight]';
    }else{
      _bmiImage = 'assets/images/normal.png';
      _bmiOutput = '${_bmi.toStringAsFixed(2)} [Normal]';
    }
  });
}
```

11. Next, insert a function named **_resetScreen** to reset state all widgets.

```
//TODO 4: Reset all widgets
void _resetScreen(){
  _weightCtrl.clear();
  _heightCtrl.clear();
  setState(() {
    _bmi = 0.0;
    _bmiOutput = '';
    _bmiImage = 'assets/images/empty.png';
  });
}
```

The setState() method is crucial for updating the UI in a stateful widget. When you call setState(), it tells Flutter that the widget's state has changed.

12. Within the **_MyHomePageState**, override the dispose method. You may override any class method by typing **@o** and then select the **dispose** method.



All **TextEditingController** should be disposed within the dispose method. The **dispose** method is called when the user is leaving your app. Controllers hold references to the text

they manage. Dispose these controllers avoid memory leaks, which can prevent performance issues or even app crashes.

Complete the following codes:

```
//TODO 5: Override the dispose method
@override
 void dispose() {
   super.dispose();
   _weightCtrl.dispose();
   _heightCtrl.dispose();
}
```

13. Within the Scaffold widget, insert a new attribute named **resizeToAvoidBottomInset** and set it to **false**.

```
return Scaffold(
  //TODO 6: set resize of screen to avoid overflow issue
  resizeToAvoidBottomInset: false,
  appBar: AppBar(
    backgroundColor: Theme.of(context).colorScheme.inversePrimary,
    title: Text(widget.title),
  ),
```

14. Wrap the **Column** widget with a **Padding** widget, which adds empty space around its child. Within the **Padding** widget:

```
body: Center(
  //TODO 7: Insert a Padding widget
  child: Padding(
    padding: const EdgeInsets.all(8.0),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
      ...
```

15. Within the children of the **Column** widget, insert a **Stack** widget.

```
    //TODO 8: Insert Stack with image and text
    Stack(
      fit: StackFit.loose,
      alignment: AlignmentDirectional.center,
      children: [
       //TODO 9: Insert two containers
      ],
),
```

16. Within the Stack widget, insert two Container widgets. The first container holds an Image widget that represents the BMI status.

```
Container(
  width: 160,
  height: 160,
  decoration: BoxDecoration(
    border: Border.all(
      color: Colors.orange,
      width: 3,
    ),
    borderRadius: BorderRadius.circular(10.0),
```

```
    ),
    child: Image.asset(
      _bmiImage,
    ),
  ),
```

The second container holds a **Text** widget that displays instructions guiding users on how to operate the app.

```
Container(
  width: 150,
  height: 150,
  alignment: Alignment.center,
  child: _bmi==0.0 ? Text(
    textAlign: TextAlign.center,
    'Enter body weight and height to know your Body Mass Index (BMI)',
    style: TextStyle(fontSize: 12, color: Colors.orange, ),
  ): Text(''),
),
```

Notice that the second Text widget displays text conditionally based on the value of a variable named **_bmi**. In this case, the code displays a message prompting the user to enter their weight and height. Once the user enters the values and the BMI is calculated, _bmi will no longer be 0.0, and the empty Text widget will be displayed, effectively hiding the initial message. Save and run your app, you should see the following outputs:

17. Continue to insert two Text widgets. These are to display BMI value and the status.

```
//TODO 10: Insert text widget showing BMI status
const Text(
  'You Body Mass Index (BMI) is :',
),
Text(
  _bmiOutput,
  style: Theme.of(context).textTheme.displaySmall,
),
```

18. Continue to insert two TextField widgets, which are to capture inputs from the user.

```
//TODO 11: Insert TextField and Button
TextField(
  controller: _weightCtrl,
  keyboardType: TextInputType.number,
  decoration: InputDecoration(
    labelText: 'Enter weight (kg)',
  ),
),
TextField(
  controller: _heightCtrl,
  keyboardType: TextInputType.number,
  decoration: InputDecoration(
    labelText: 'Enter height (meter)',
  ),
),
```

Setting the **keyboardType** to **TextInputType.number** within the TextField widget restricts the user input to numbers.

19. Insert an Expanded widget that takes up empty screen spaces and pushes the subsequent widget to the bottom of the screen.

```
//TODO 12: Insert an Expanded widget to take up empty screen spaces
Expanded(child: SizedBox(height: double.infinity,)),
```

20. Finally, insert a Row widget, which has two buttons. The Row widget is used to arrange the buttons horizontally.

```
//TODO 13: Insert reset and calculate buttons
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    ElevatedButton(
        onPressed: _resetScreen, child: Text('Reset')),
    ElevatedButton(
        onPressed: _calculateBMI, child: Text('Calculate')),
  ],
),
```

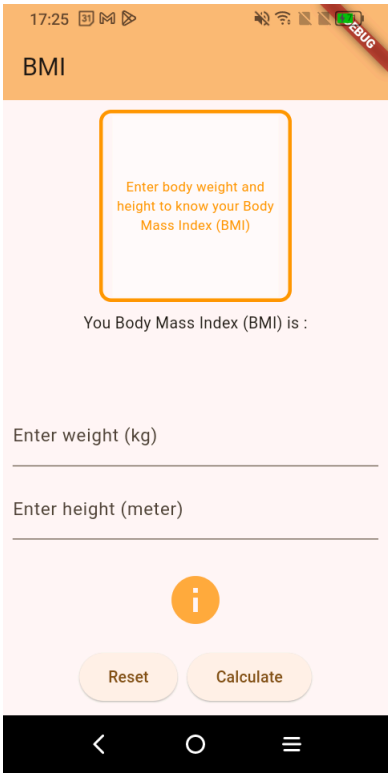
Save and run your app.

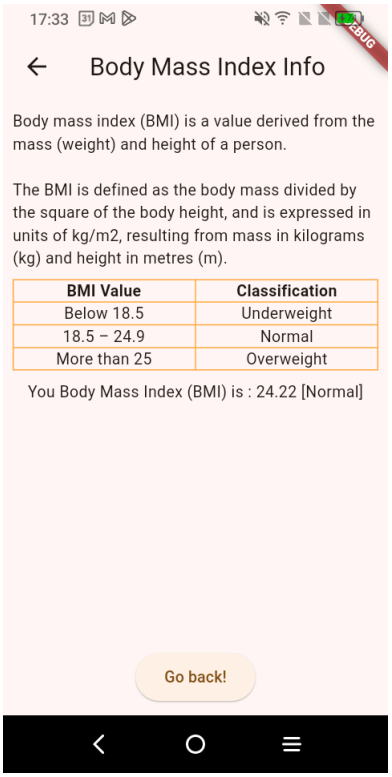
TODO: Enhance the app by including input validation. Implement range field validation, ensuring that input falls within an acceptable range of values (e.g. age between 0 and 100).

Practical 4: Navigation

We will learn the basic of Flutter navigation in this practical. It allows us to move from one screen to another. Also, we will pass a piece of data from the first screen to the second screen.

We will enhance the BMI calculator with a new Information screen, showing a short description about BMI, the formula to calculate it, and a list of BMI values. Also, we will pass the value of BMI from the main to the information screen.





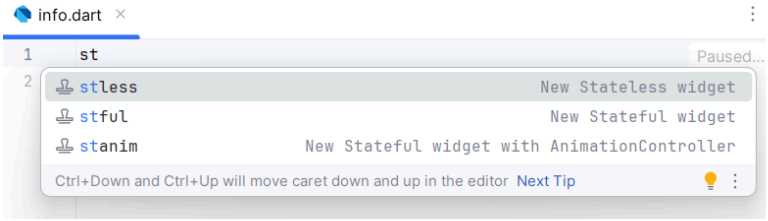
1. Open the BMI project.

2. Modify the **main.dart** file. Insert a new **IconButton** below the **Expanded** widget:

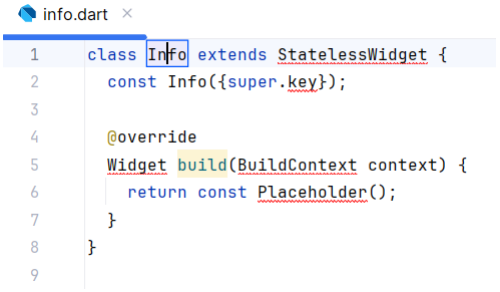
```
//TODO 12: Insert an Expanded widget to take up empty screen spaces
Expanded(child: SizedBox(height: double.infinity,)),

IconButton(
  icon: Icon(Icons.info),
  iconSize: 48,
  color: Colors.orangeAccent,
  onPressed: () {
    Navigator.push(
        context,
        MaterialPageRoute(builder: (context) => Info(bmi: _bmiOutput,)));
  },
),
```

3. Right-click on the **lib** folder, select **New** -> **Dart File**. Assign the name **info.dart** to the new file.

4. Within the **info.dart** file, enter **stl** and the system will prompt you a list of widgets, select **stless** to create a new **Stateless** widget. We will not implement any UI update here, therefore, a Stateless widget is appropriate.



Assign Info as the name of the newly created stateless widget:



5. Import the following package:

```
import 'package:flutter/material.dart';
```

6. Within the **Info** class, define a local variable named **bmi** to hold the BMI data passing from the first screen. Also, modify the Info constructor to include the **bmi** value as a **required** value.

```
class Info extends StatelessWidget {
  //TODO 1: Define a local variable to accept value
  final String bmi;
  const Info({super.key, required this.bmi});
```

7. Replace the **Placeholder** widget with a **Scaffold** widget. Also, assign a Text to the widget showing the heading text.

```
//TODO 2: Replace the Placeholder with a Scaffold
return Scaffold(
  appBar: AppBar(
    title: const Text('Body Mass Index Info'),
  ),
```

8. Within the **body** of the Scaffold, insert a Padding, a Center, and a Column widget.

```
//TODO 3: Insert Padding, Center, and Column
body: Padding(
  padding: const EdgeInsets.all(8.0),
  child: Center(
    child: Column(
      children: [],
    ), //Column
  ), //Center
), //Padding
```

9. Inside the **children** of the **Column**, insert the following widgets:
   a. A Text widget to show the general description of BMI.

```
Text('Body mass index (BMI) is a value derived from the mass (weight) and height of
a person.\n\nThe BMI is defined as the body mass divided by the square of the body
height, and is expressed in units of kg/m\u00B2, resulting from mass in kilograms
(kg) and height in metres (m).'),
```

   b. A Table widget to show BMI value and classification.

```
Table(
  border: TableBorder.all(color: Colors.orangeAccent),
  //defaultVerticalAlignment: TableCellVerticalAlignment.middle,
  children: const [
    TableRow(
      children: [
        TableCell(child: Text('BMI Value',
            textAlign: TextAlign.center,
            style: TextStyle(fontWeight: FontWeight.bold),)),
        TableCell(child: Text('Classification',
            textAlign: TextAlign.center,
            style: TextStyle(fontWeight: FontWeight.bold))),
      ],
    TableRow(
      children: [
        TableCell(child: Center(child: Text('Below 18.5',
            textAlign: TextAlign.center,))),
        TableCell(child: Center(child: Text('Underweight',
            textAlign: TextAlign.center,))),
      ],
    ),
      //TODO: Continue inserting the remaining two rows.
      ...
    ],
  ),
```

   c. A Text widget to show the BMI value received from the first screen.

```
bmi==''? Text('Please enter your weight and height.'):
           Text('You Body Mass Index (BMI) is : $bmi',),
```

   d. An Expanded widget.

```
Expanded(child: SizedBox()),
```

   e. A ElevatedButton widget.

```
ElevatedButton(
  onPressed: () {
    Navigator.pop(context);
  },
  child: const Text('Go back!'),
),
```

10. Save and run the project.

TODO: Enhance the app by including a link to a web URL at https://data.worldobesity.org/country/malaysia-130/

Practical 5: State Management

State management refers to the management and manipulation of data within an app to ensure that the user interface (UI) accurately reflects the current state of the application. This lab utilizes the Provider package, which allows you to hold application's state.

1. We are going to create a mobile app that utilizing the **Provider** state management as below:

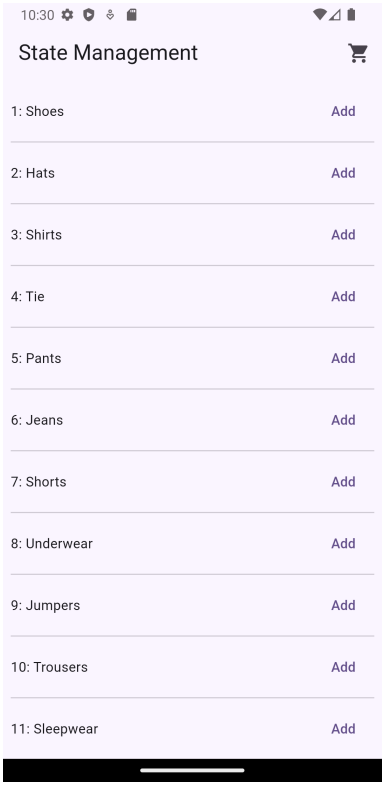


Figure 1 Item List

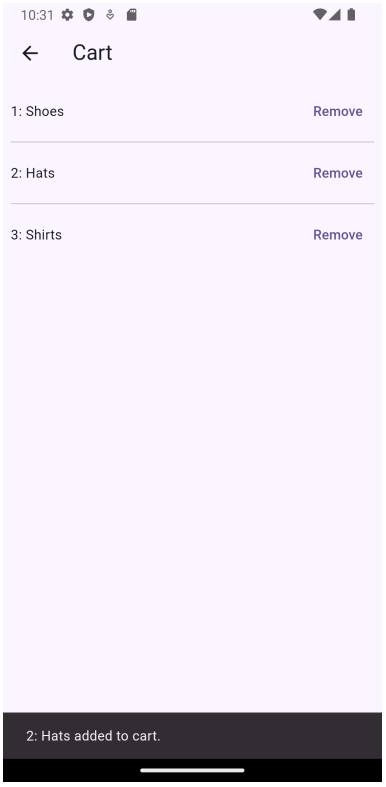


Figure 2 Cart

1. Create a new Flutter project using the **Application** template.

2. Open the **Terminal** and enter the following command:

```
flutter pub add provider
```

3. Create a new dart file named **item.dart**. This is a model for items to be stored in the memory.

```dart
class Item{
  int id;
  String name;
  //Constructor with initializing formal parameters
  Item(this.id, this.name);
  int get itemID => id;
  String get itemName => name;
  @override
  String toString() {
    return '$id: $name';
  }
}
```

4. Create a new dart file named **cart_provider.dart**. This class holds a list of items (the cart) that could be accessed by all the screens/routes within the app.

```
import 'package:flutter/material.dart';
import 'item.dart';

//Providing change notification to its listeners
class CartProvider extends ChangeNotifier {
  final List<Item> itemList = [];

  void add(Item item){
    itemList.add(item);
    notifyListeners();
  }

  void remove(Item item){
    itemList.remove(item);
    notifyListeners();
  }
}
```

5. In the **main.dart** file, observe the dummy data list as follows: -

```
final List<Item> catalog = [
  Item(1, 'Shoes'),
  Item(2, 'Hats'),
  Item(3, 'Shirts'),
  Item(4, 'Tie'),
  Item(5, 'Pants'),
  Item(6, 'Jeans'),
  Item(7, 'Shorts'),
  Item(8, 'Underwear'),
  Item(9, 'Jumpers'),
  Item(10, 'Trousers'),
  Item(11, 'Sleepwear'),
  Item(12, 'Accessories'),
];
```

This catalog list should be declared as a global variable, which can be above the main() function.

6. In the **main.dart** file, update the main() function to include an instance of a **ChangeNotifer** as follows:-

```
void main() {
  //TODO - The ChangeNotifierProvider provides an instance of a ChangeNotifier to
its descendants
  runApp(
    ChangeNotifierProvider(
      create: (context) => CartProvider(),
      child: const MyApp(),
    ),
  );
}
```

7. In the **main.dart** file, insert a new class named **CartItem**, which is used to present an item in a list. Each cart item has an id, name, and an Add button:

```
class CartItem extends StatelessWidget {
  final int index;

  const CartItem({super.key, required this.index});

  @override
  Widget build(BuildContext context) {
    var item = catalog[index];
    return Row(
      children: [
        //TODO - Insert widgets for each row
        Text('$item'),
        const Expanded(child: SizedBox()),
        Consumer<CartProvider>(builder: (context, cart, child) {
          return TextButton(
              onPressed: () {
                cart.add(item);
                ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(
                      content: Text('$item added to cart.'),
                    ),
                );
              },
              child: const Text('Add'));
        }),
      ],
    );
  }
```

The body section contains a **Consumer** that can access a **ChangeNotifer** named **CartProvider**.

8. Modify the body section of the **`build()`** method of the **_MyHomePageState** to include a **`ListView`** as follows:-

```
body: Center(
    child: ListView.separated(
        padding: const EdgeInsets.all(8),
        scrollDirection: Axis.vertical,
        shrinkWrap: true,
        itemBuilder: (BuildContext context, int index) {
          return CartItem(index: index);
        },
        separatorBuilder: (BuildContext context, int index) =>
            const Divider(),
        itemCount: catalog.length
    )
),
```

9. Run the project. You should be able to see a ListView as shown in Figure 1 Item List.

10. In the **main.dart** file, **MyAppState's build()** method, include an icon button in the **appBar** section:

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('State Management'),
```

```
      actions: [
        IconButton(
          onPressed: () {
            Navigator.push<void>(
                context,
                MaterialPageRoute<void>(
                  builder: (BuildContext context) => const CartPage()));
          },
          icon: const Icon(Icons.shopping_cart),
        ),
      ],
    ),
```

Ignore the error for CartPage(), we will create it later.

11. Create a new dart file named **card_dart** as follows: -

```
import 'package:demo_state_mgmt/cart_provider.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'item.dart';

class CartPage extends StatelessWidget {
  const CartPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Cart'),
      ),
      body: const Column(
        mainAxisAlignment: MainAxisAlignment.start,
        children: [
          CartListWidget(),
        ],
      ),
    );
  }
}

class CartListWidget extends StatefulWidget {
  const CartListWidget({super.key});

  @override
  State<CartListWidget> createState() => _CartListWidgetState();
}

class _CartListWidgetState extends State<CartListWidget> {
  @override
  Widget build(BuildContext context) {
    return Consumer<CartProvider>(
        builder: (context, cart, child) {
            return cart.itemList.isNotEmpty
            ? ListView.separated(
              padding: const EdgeInsets.all(8),
              scrollDirection: Axis.vertical,
              shrinkWrap: true,
              itemBuilder: (BuildContext context, int index){
                return CartItem(item: cart.itemList[index]);
              },
              itemCount: cart.itemList.length,
              separatorBuilder: (BuildContext context, int index) => const
Divider(),
```

```
            )
            : const Center(child: Text('Cart is empty.'));
        }
    );
  }
}


class CartItem extends StatelessWidget {
  final Item item;

  const CartItem({super.key, required this.item});

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Text('$item'),
        const Expanded(child: SizedBox()),
        TextButton(
            onPressed: () {
              Provider.of<CartProvider>(context, listen: false).remove(item);
            },
            child: const Text('Remove')),
      ],
    );
  }
}
```

This widget is similar to the main.dart's MyApp() widget, however, the item list has a remove button.

12. Save and run the project. Try adding some items to the shopping list and then click the cart button.

TODO: Enhance the app by including a cart item counter displayed at the menu bar.

Practical 6: Form and Input Validation

The Form widget in Flutter is a fundamental widget for building forms. It provides a way to group multiple form fields together, perform validation on those fields, and manage their state. This lab demonstrates the use of TextFormField, Checkbox, RadioList, and DropdownButton widgets. We will create a car loan calculator.

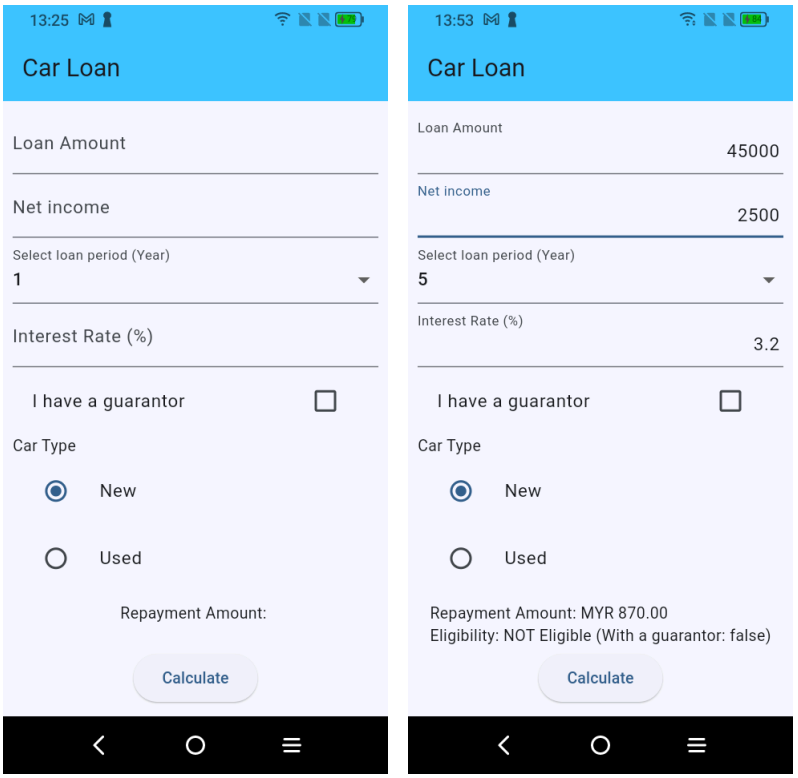
Formulas given are as follows: -

a. Interest = loan amount * (interest rate/100) * loan period
b. Repayment amount = (loan amount + interest) / (loan period * 12 months)

For example, if the loan amount is RM 45,000 with an interest rate of 3.2% for 5 years.

Interest = 3.2% x RM 45,000 x 5 years = RM 7,200
Repayment amount = (RM 45,000 + RM 7,200) / (5 years X 12 months) = RM 870

If the repayment amount is more than 30% of a person's net income, then the loan application should be rejected, unless the person has a guarantor.



1. Create a new Flutter project type **Application** name car_loan. Set the package name to my.edu.tarc.

2. Open the terminal window and enter the following command to include the internationalization package:

```
flutter pub add flutter_localizations --sdk=flutter
flutter pub add intl:any
```

3. Open the **main.dart** file and include the following package:

```
import 'package:intl/intl.dart' as intl;
```

4. Change the **title** attribute of **MaterialApp** and **MyHomePage** to 'Car Loan'.

5. Within the **_MyHomePageState** class, declare some controllers and local variables:

```
//Declare local variables
double _loanAmount = 0.0;
double _netIncome = 0.0;
double _interestRate = 0.0;
int _loanPeriod = 1;
bool _hasGuarantor = false;
int _carType = 1; //1 = New, 2 = Used
double _repaymentAmount = 0.0;
String _repaymentOutput = '';
final _years = [1,2,3,4,5,6,7,8,9];

//Controller
final loanAmountCtrl = TextEditingController();
final netIncomeCtrl = TextEditingController();
final interestRateCtrl = TextEditingController();
//Set focus to a specific widget
final _myFocusNode = FocusNode();

//Format output with the currency symbol of Malaysia
final myCurrency = intl.NumberFormat('#,##0.00', 'ms_MY');

//Form controller - manages the overall form state
final _formKey = GlobalKey<FormState>();
```

The **_formKey** is used to access the FormState object, which allows you to validate and save the form.

6. The app shall prompt a simple dialog if the user is not eligible for a car loan. Continue in the **_MyHomePageState** class, we create a method to show the dialog:

```
void myAlertDialog(){
  AlertDialog eligibilityAlertDialog = AlertDialog(
    title: const Text('Eligibility'),
    content: const Text('You are not eligible for this loan. '
        'Get a guarantor to proceed'),
    actions: [
      TextButton(
          onPressed: (){
            Navigator.pop(context);
          },
          child: const Text('Ok')),
    ],
  );
  showDialog(
      context: context,
      builder: (BuildContext context){
        return eligibilityAlertDialog;
      });
}
```

7. Within the **_MyHomePageState** class, create a method to calculate the repayment amount:

```
void _calculateRepayment(){
  _loanAmount = double.parse(loanAmountCtrl.text);
  _netIncome = double.parse(netIncomeCtrl.text);
  _interestRate = double.parse(interestRateCtrl.text);
  var interest = _loanAmount * _loanPeriod * (_interestRate/100);
  _repaymentAmount = (_loanAmount + interest) / (_loanPeriod * 12);
  bool eligible = _netIncome * 0.3 >= _repaymentAmount;
  if(eligible || _hasGuarantor){
    setState(() {
      _repaymentOutput = 'Repayment Amount : '
          '${myCurrency.currencySymbol} '
          '${myCurrency.format(_repaymentAmount)} '
          '\n '
          'Eligibility : ${eligible? 'Eligible': 'Not Eligible'}';
    });
  }else{
    myAlertDialog();
  }
}
```

8. You may remove the following codes from the program:

```
int _counter = 0;

void _incrementCounter() {
  setState(() {
    _counter++;
  });
}
```

Also, you may remove the **FloatingActionButton**.

9. In the **build** function of **_MyHomePageState** class, wrap the Center widget with a Form widget. Assign **_formKey** to the **key** attribute.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      title: Text(widget.title),
    ),
    body: Form(
      key: _formKey,
      child: Center(
        child: Column(
        ...
```

10. Within the **children** of the **Column** widget, define two **TextFormField** widgets.

```
TextFormField(
  decoration: const InputDecoration(
    labelText: 'Loan Amount',
  ),
  keyboardType: TextInputType.number,
  inputFormatters: [FilteringTextInputFormatter.digitsOnly],
  controller: loanAmountCtrl,
  focusNode: _myFocusNode,
  validator: (value){
    if(value == null || value.isEmpty){
      return 'Please enter loan amount';
```

```
      }
      return null;
    },
  ),
  TextFormField(
    decoration: const InputDecoration(
      labelText: 'Net Income',
    ),
    keyboardType: TextInputType.number,
    inputFormatters: [FilteringTextInputFormatter.digitsOnly],
    controller: netIncomeCtrl,
    validator: (value){
      if(value == null || value.isEmpty){
        return 'Please enter net income';
      }
      return null;
    },
  ),
```

Notice that each **TextFormField** widget has a **validator**. This function checks if the input is valid. If the input is invalid, it returns an error message. If the input is valid, it returns null.

11. Continue with UI with a **DropdownButtonFormField** widget.

```
DropdownButtonFormField(
    value: _loanPeriod,
    items: _years.map((int item){
      return DropdownMenuItem(
        value: item,
        child: Text('$item year(s)'),
      );
    }).toList(),
    onChanged: (int? item){
      setState(() {
        _loanPeriod = item!;
      });
    },
  validator: (value){
      if(value == 0){
        return 'Please select an option';
      }
      return null;
  },
  decoration: const InputDecoration(
    labelText: 'Select loan period (year)'
  ),
),
```

12. Next, insert a TextFormField to accept the interest rate.

```
TextFormField(
  keyboardType: const TextInputType.numberWithOptions(
    decimal: true,
    signed: false
  ),
  inputFormatters: [
    FilteringTextInputFormatter.allow(RegExp(r'[0-9.]'))
  ],
  decoration: const InputDecoration(
    labelText: 'Interest Rate (%)'
  ),
  controller: interestRateCtrl,
  validator: (value){
```

```
      if(value == null || value.isEmpty){
        return 'Please enter interest rate';
      }
      return null;
    },
  ),
```

This field applies a regular expression, which ensure the interest rate is in the right format. Find out more about Regular Expression here:
https://api.flutter.dev/flutter/dart-core/RegExp-class.html

13. We also insert a **CheckBoxTitle** widget for the user to indicate the status of guarantor.

```
CheckboxListTile(
    value: _hasGuarantor,
    title: const Text('I have a guarantor'),
    onChanged: (value) {
      setState(() {
        _hasGuarantor = value!;
      });
    }),
```

14. Next, insert a Text and two RadioListTile widgets.

```
const Align(
  alignment: Alignment.centerLeft,
  child: Text('Car Type', textDirection: TextDirection.ltr,),
),
RadioListTile(
    title: const Text('New'),
    value: 1, //New car
    groupValue: _carType,
    onChanged: (value){
      setState(() {
        _carType = value!;
      });
    }),
RadioListTile(
    title: const Text('Used'),
    value: 2, //Used car
    groupValue: _carType,
    onChanged: (value){
      setState(() {
        _carType = value!;
      });
    }),
```

For the two **RadioListTile** to act as a cohesive group, all widgets should be assigned the same **groupValue**, but each widget should be assigned with a unique **value**.

15. Insert a Text and an ElevatedButton widgets.

```
// Display repayment amount
Text(_repaymentOutput),


ElevatedButton(
    onPressed: () {
      // Validate returns true if the form is valid, or false otherwise.
      if(_formKey.currentState!.validate()){
```

```
            // If the form is valid
            if(validInterest(_carType)){
              _calculateRepayment();
            }else{
              ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(content: Text('Invalid interest rate'),)
              );
            }
          }
        },
        child: const Text('Calculate')
),
```

16. Finally, override the dispose method.

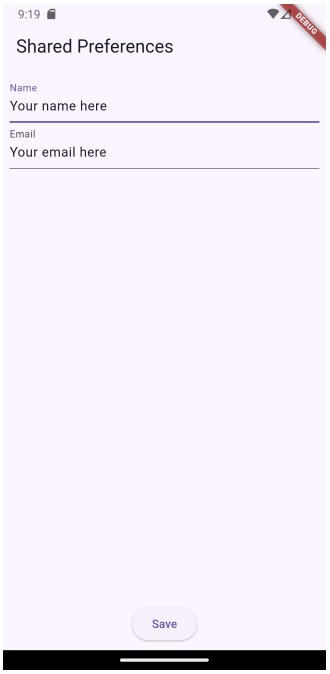
```
@override void dispose() {
  // TODO: implement dispose
  super.dispose();
  loanAmountCtrl.dispose();
  netIncomeCtrl.dispose();
  interestRateCtrl.dispose();
}
```

When the **Form** widget is disposed of, the form state is also disposed of, and any associated resources are released. Since Form is a widget, we do not need to dispose the validator.

TODO: Assume that for new cars, you can get a fixed interest rate of 2.5% to 3.2% p.a. Used cars, on the other hand, come with an interest rate of 3.2% to 4.45% p.a. Implement the necessary input validation.

Practical 7: Shared Preferences

1. Create a new Flutter project type **Application** name **user_profile**. Set the package name to **my.edu.tarc**. This app utilises the **shared_preferences** library to store key-value pairs in local storage. For this case, we will save name and email address of a user as shown below:



2. Open a new **Terminal** and enter the following command:

```
flutter pub add shared_preferences
```

3. Open the **main.dart** file and import the shared_preferences.dart file:

```
//TODO 1: - Import reference to async and shared_preferences
import 'dart:async';
import 'package:shared_preferences/shared_preferences.dart';
```

4. In the class _MyHomePageState extends State<MyHomePage>, insert TextEditingController for the two TextField widgets in the class:

```
//TODO 2: - Insert TextEditingController
final _nameTextEditingController = TextEditingController();
final _emailTextEditingController = TextEditingController();
```

5. Within the class class SharedPreferencesDemoState extends State<SharedPreferencesDemo>, insert to asynchronies functions as follows: -

```
//TODO 3: - Insert _loadProfile() and _updateProfile() functions
Future<void> _loadProfile() async{
  final prefs = await SharedPreferences.getInstance();
  _nameTextEditingController.text = prefs.getString('name')?? "";
  _emailTextEditingController.text = prefs.getString('email') ?? "";
}
```

```
Future<void> _updateProfile() async{
  final prefs = await SharedPreferences.getInstance();
  prefs.setString('name', _nameTextEditingController.text);
  prefs.setString('email', _emailTextEditingController.text);
}
```

6. Override the iniState() and dispose() functions as follows:

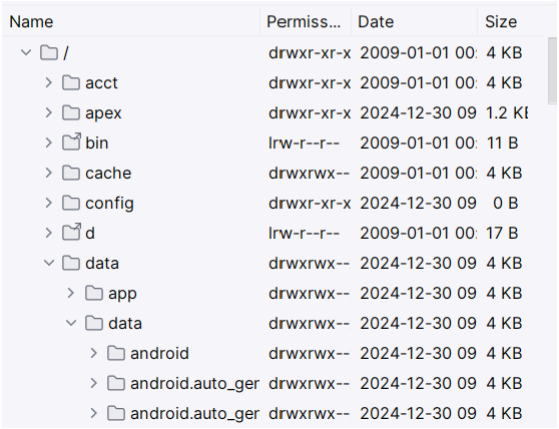
```
//TODO 4:- Override initState() and dispose() functions
@override
void initState() {
  _loadProfile();
  super.initState();
}

@override
void dispose() {
  super.dispose();
  _nameTextEditingController.dispose();
  _emailTextEditingController.dispose();
}
```

7. In the **body** of the **build()** function, create two **TextField** widgets and a **ElevatedButton** widget.

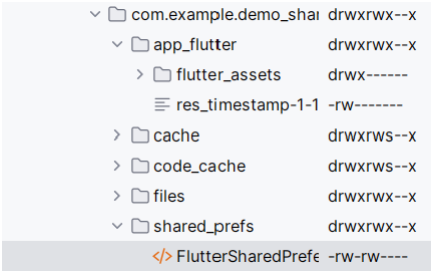
```
//TODO 5: - Insert TextField and ElevatedButton
TextField(
  controller: _nameTextEditingController,
  keyboardType: TextInputType.name,
  decoration: const InputDecoration(
    labelText: 'Name',
  ),

),
const SizedBox(),
TextField(
  controller: _emailTextEditingController,
  keyboardType: TextInputType.emailAddress,
  decoration: const InputDecoration(
    labelText: 'Email',
  ),
),
const Expanded(child: SizedBox()),
ElevatedButton(
    onPressed: (){
      _updateProfile();
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Profile info saved.'),
        ),
      );
    },
    child: const Text('Save')
),
```

8. Run the app, insert a name and an email, press the **Save** button.

9. On the main menu, select **View** -> **Tool Windows** ->**Device Explorer**. In the Device Explorer, click the **data** -> **data** folder.



10. Scroll down and look for your application package name. Inside the **shared_prefs** folder, you will find a shared preference file titled **FlutterSharedPreferences.xml**.



11. Open the file to observe its contents.

Congratulations! Your app can now save data to the shared preference file.

TODO: Enhance the app to ensure all inputs are in the right format.

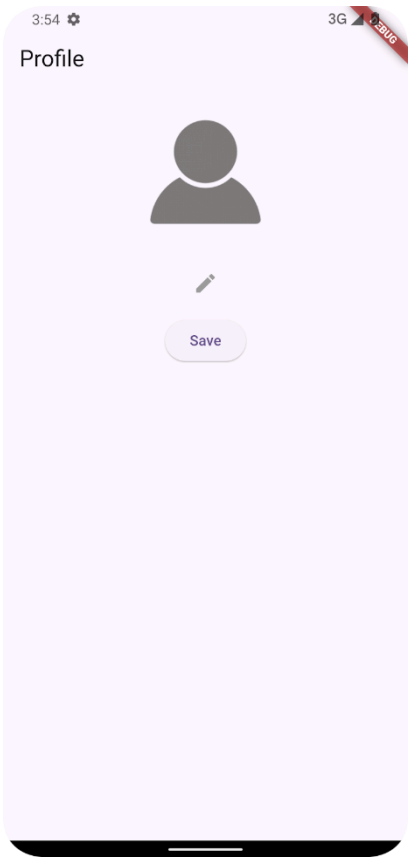
Name: ensure that name only contains letters, spaces, and potentially certain punctuation marks such as hyphens or apostrophes.

Email: A valid email address consists of an email prefix and an email domain, both in acceptable formats.

Practical 8: Data File

This app allows users to pick an image from the Gallery and save it as a profile picture in a folder dedicated to the app. This app utilizes the image picker and file Input/Output operations.

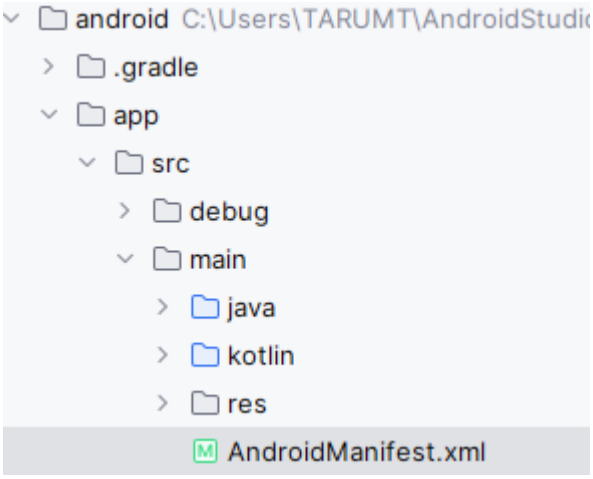
1. Create a new Flutter app using the Application template.



Default profile picture

Note: Download a profile image and insert it to the assets folder of your app.

2. In the project File Explorer, open the Insert the following permission into the Android'smanifest file.



```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <application
        android:label="helloflutter"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
```

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

3. In the terminal, run the following command

```
flutter pub add image_picker path_provider
```

4. Open the **main.dart** file. Import the following packages:

```
import 'package:image_picker/image_picker.dart';
import 'package:path_provider/path_provider.dart';
```

5. Within the stateful widget, declare a file and an image picker instance:

```
//TODO - Create a file and an image picker instances
File? _image;
final picker = ImagePicker();
```

6. Create a method to obtain an image file from the Gallery:

```
//TODO - Obtain profile image from the Gallery
Future getImageFromGallery() async {
  final pickedFile = await picker.pickImage(source: ImageSource.gallery);

  if (pickedFile != null) {
    setState(() {
      _image = File(pickedFile.path);
    });
  }
}
```

7. Create a method to save the profile image into the app folder:

```
//TODO - Save the profile image into the app folder
Future<void> savePicture() async {
  if (_image != null) {
    try {
      // Save the image to a specific location
      final appDocDir = await getApplicationDocumentsDirectory();
      final newImagePath = '${appDocDir.path}/profile.png';
      await _image!.copy(newImagePath);
      print('File image copied successfully to $newImagePath');
    } catch (e) {
      print('File error copying image: $e');
    }
  } else {
    AlertDialog(
      title: const Text('Profile Image'),
      content: const SingleChildScrollView(
        child: ListBody(
          children: <Widget>[
            Text('Profile Image'),
            Text('Profile Image file is missing.'),
```

```
            ],
          ),
        ),
        actions: <Widget>[
          TextButton(
            child: const Text('Close'),
            onPressed: () {
              Navigator.of(context).pop();
            },
          ),
        ],
      );
    }
  }
```

8. Create a method to load the profile image:

```
//TODO - Load profile image from the app folder
Future<void> loadProfileImage() async {
  // Get the application documents directory
  final appDocDir = await getApplicationDocumentsDirectory();
  final imagePath = '${appDocDir.path}/profile.png';

  // Create the destination file path
  final file = File(imagePath);

  if (await file.exists()) {
    setState(() {
      _image = file;
      print('File path: $imagePath');
    });
  } else {
    print('File not found in $imagePath');
  }
}
```

9. Call the loadProfileImage method in the initState method:

```
@override
void initState() {
  super.initState();
  //TODO - Call the loadProfileImage method during app initialization stage
  loadProfileImage();
}
```

10. Construct the main UI:

```
//TODO - Construct the main UI
_image == null
    ? Image.asset(
        'assets/profile.png',
        width: 150,
        height: 150,
      )
    : Image.file(
        _image!,
        width: 150,
        height: 150,
      ),
const SizedBox(height: 8.0,),
IconButton(
```

```
  icon: const Icon(Icons.edit),
  color: Colors.grey,
  onPressed: () {
    getImageFromGallery();
  },
),
const SizedBox(height: 8.0,),
ElevatedButton(
    onPressed: () {
      savePicture();
      ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Profile image saved.')));
    },
    child: const Text('Save'),
),
```

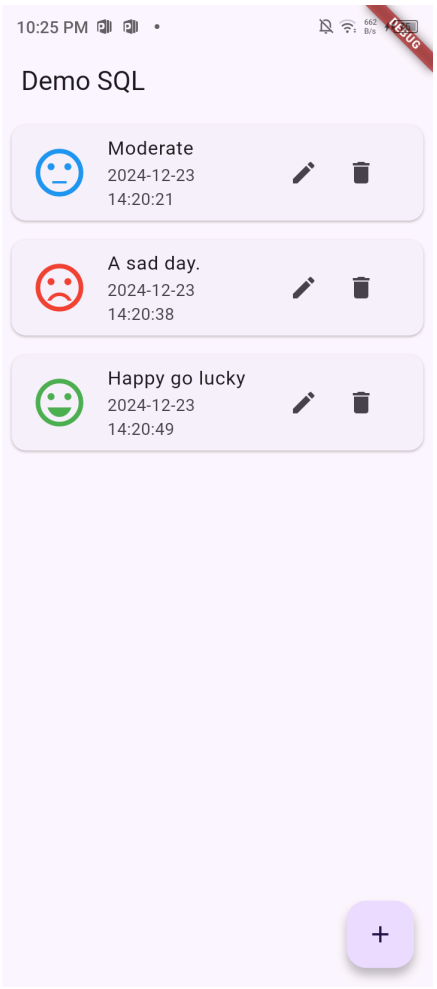
11. Run the app. Pick an image from the Gallery and hit the Save button.

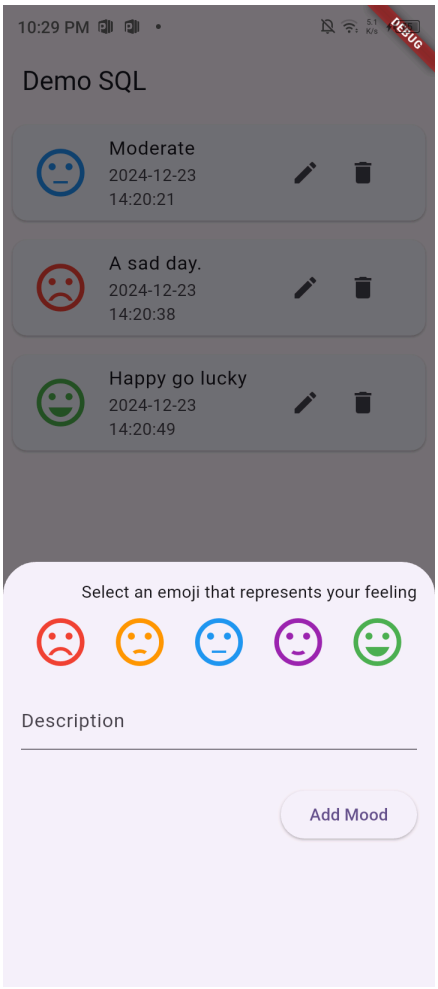
12. Click the menu **View** -> **Tool Windows** -> **Device Explorer**. Expand the folder data -> data >-<your_app_package_name> -> app_flutter, you should be able to find the image file named profile.png. Congratulations! You have completed this practical.

TODO: Enhance the app by reducing the image size – apply an image compression library.

Practical 9: SQLite

1. Create a new Flutter app using the Application template. It is a mood journal app allowing users to perform emotion tracking.





2. In the Terminal, add the following dependencies:

```
flutter pub add sqflite

flutter pub add uuid

flutter pub add path_provider
```

3. Insert a new dart file named mood_model.dart. This is a model class to perform CRUD operations of mood records.

```
class MoodModel{
  final int id;
  final int scale;
  final String description;
  final String createdOn;
```

```
  MoodModel({
    required this.id,
    required this.scale,
    required this.description,
    required this.createdOn,
  });

  factory MoodModel.fromJson(Map<String, dynamic>data) => MoodModel(
      id: data['id'],
      scale: data['scale'],
      description: data['description'],
      createdOn: data['createdOn'],
  );

  Map<String, dynamic> toMap() => {
    'id': id,
    'scale': scale,
    'description': description,
    'createdOn': createdOn,
  };
}
```

4. Insert a new dart file named database_service.dart. This class performs all the database operations.

```
import 'dart:developer';
import 'package:demo_sqlite/mood_model.dart';
import 'package:path_provider/path_provider.dart';
import 'package:sqflite/sqflite.dart';

class DatabaseService{
  static final DatabaseService _databaseService = DatabaseService._internal();
  factory DatabaseService() => _databaseService;
  DatabaseService._internal();
  static Database? _database;

  //Get an instance of database
  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await initDatabase();
    return _database!;
  }


  //Initialize a database
  Future<Database> initDatabase() async {
    final getDirectory = await getApplicationDocumentsDirectory();
    String path = '${getDirectory.path}/moods.db';
    log(path);
    return await openDatabase(path, onCreate: _onCreate, version: 1);
  }
```

```
  //Create an instance of database
  void _onCreate(Database db, int version) async {
    await db.execute(
        'CREATE TABLE Moods('
            'id INTEGER PRIMARY KEY AUTOINCREMENT, '
            'scale INTEGER, '
            'description TEXT, '
            'createdOn DATETIME DEFAULT CURRENT_TIMESTAMP)');
    log('TABLE CREATED');
  }

  Future<List<MoodModel>> getMood() async {
    final db = await _databaseService.database;
    var data = await db.query('Moods');
    List<MoodModel> moods =
    List.generate(data.length, (index) => MoodModel.fromJson(data[index]));
    print(moods.length);
    return moods;
  }

  Future<void> insertMood(MoodModel mood) async {
    final db = await _databaseService.database;
    var data = await db.rawInsert(
        'INSERT INTO Moods(scale, description) VALUES(?,?)',
        [mood.scale, mood.description]);
    log('inserted $data');
  }

  Future<void> editMood(MoodModel mood) async {
    final db = await _databaseService.database;
    var data = await db.update('Moods', mood.toMap(), where: 'id=?', whereArgs:
[mood.id]);
    log('updated $data');
  }


  Future<void> deleteMood(int id) async {
    final db = await _databaseService.database;
    var data = await db.delete('Moods', where: 'id = ?', whereArgs: [id]);
    log('deleted $data');
  }
}
```

5. In the **main.dart** file, within the Stateful widget class:

```
//TODO - Declare instance of database service and local variables
final dbService = DatabaseService();
final descriptionController = TextEditingController();
int _scale = 3;
```

6. In the **main.dart** file, within the Stateful widget class, insert a function named showBottomSheet. This function will prompt a Modal Bottom Sheet, which users can insert their mood journals.

```
//TODO - Insert showBottomSheet

void showBottomSheet(String functionTitle, Function()? onPressed) {
  showModalBottomSheet(
      context: context,
      elevation: 5,
      isScrollControlled: true,
      builder: (_) => Container(
            padding: EdgeInsets.only(
              top: 15, left: 15, right: 15,
              bottom: MediaQuery.of(context).viewInsets.bottom + 120,
            ),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              crossAxisAlignment: CrossAxisAlignment.end,
              children: [
                Text('Select an emoji that represents your feeling',),
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceAround,
                  children: [
                    IconButton(
                      icon: Icon(Icons.sentiment_very_dissatisfied),
                      iconSize: 48,
                      color: Colors.red,
                      onPressed: () {
                        _scale = 1;
                      },
                    ),
                    IconButton(
                      icon: Icon(Icons.sentiment_dissatisfied),
                      iconSize: 48,
                      color: Colors.orange,
                      onPressed: () {
                        _scale = 2;
                      },
                    ),
                    IconButton(
                      icon: Icon(Icons.sentiment_neutral),
                      iconSize: 48,
                      color: Colors.blue,
                      onPressed: () {
                        _scale = 3;
                      },
                    ),
                    IconButton(
                      icon: Icon(Icons.sentiment_satisfied),
                      iconSize: 48,
                      color: Colors.purple,
                      onPressed: () {
                        _scale = 4;
                      },
                    ),
                    IconButton(
                      icon: Icon(Icons.sentiment_very_satisfied),
                      iconSize: 48,
                      color: Colors.green,
                      onPressed: () {
                        _scale = 5;
                      },
                    ),
                  ],
```

```
                ),
                const SizedBox(
                  height: 10,
                ),
                TextField(
                  controller: descriptionController,
                  keyboardType: TextInputType.text,
                  decoration: const InputDecoration(
                              hintText: 'Description'),
                ),
                const SizedBox(
                  height: 10,
                ),
                const SizedBox(
                  height: 20,
                ),
                ElevatedButton(
                  onPressed: onPressed,
                  child: Text(functionTitle),
                )
              ],
            ),
          ));
}
```

7. In the main.dart file, within the Stateful widget class, insert 3 functions to add, edit, and delete emotion journals:

```
void addMood() {
  showBottomSheet('Add Mood', () async {
    var mood = MoodModel(
      id: 0,
      scale: _scale,
      description: descriptionController.text,
      createdOn: '',
    );

    dbService.insertMood(mood);
    //Update UI
    setState(() {});
    //Clear input
    descriptionController.clear();
    //Close bottom sheet
    Navigator.of(context).pop();
  });
}

void editMood(MoodModel mood) {
  descriptionController.text = mood.description;

  showBottomSheet('Update Mood', () async {
    var updatedMood = MoodModel(
      id: mood.id,
      scale: _scale,
      description: descriptionController.text,
      createdOn: mood.createdOn,
    );

    dbService.editMood(updatedMood);
    descriptionController.clear();
```

```
    setState(() {});
    Navigator.of(context).pop();
  });
}

void deleteMood(int id) {
  dbService.deleteMood(id);
  setState(() {});
}
```

8. Modify the build function of the Stateful widget:

```
//TODO - Modify the build function
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Demo SQL'),
    ),
    body: FutureBuilder<List<MoodModel>>(
        future: dbService.getMood(),
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasData) {
            if (snapshot.data!.isEmpty) {
              return const Center(
                child: Text('No record found'),
              );
            }
            return ListView.builder(
              itemCount: snapshot.data!.length,
              itemBuilder: (context, index) => Card(
                margin: const EdgeInsets.all(8),
                child: ListTile(
                    leading: emojiIcon(snapshot.data![index].scale),
                    title: Text(snapshot.data![index].description),
                    subtitle: Text(snapshot.data![index].createdOn),
                    trailing: SizedBox(
                      width: 100,
                      child: Row(
                        children: [
                          IconButton(
                            icon: const Icon(Icons.edit),
                            onPressed: () =>
                               editMood(snapshot.data![index]),
                          ),
                          IconButton(
                            icon: const Icon(Icons.delete),
                            onPressed: () =>
                                deleteMood(snapshot.data![index].id),
                          ),
                        ],
                      ),
                    )),
              ),
            );
          }
          return const Center(
            child: Text('No record found'),
          );
        }),
```

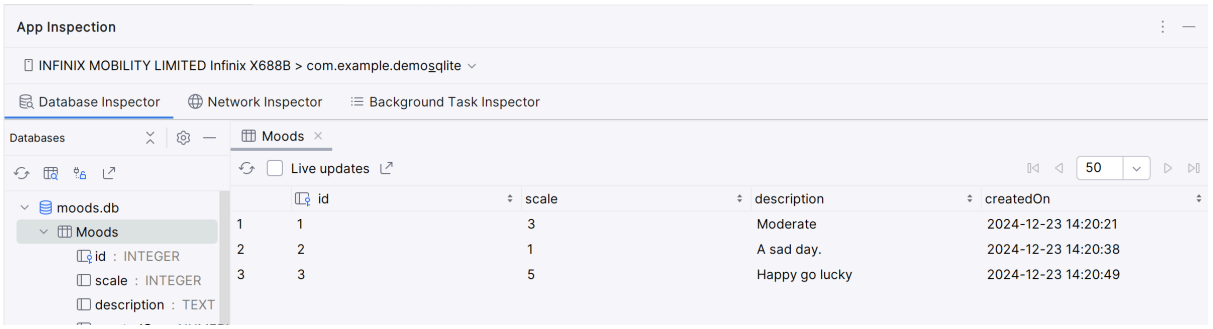
```
      floatingActionButton: FloatingActionButton(
        child: const Icon(Icons.add),
        onPressed: () => addMood(),
      ),
    );
  }
```

9. Within the main.dart class, insert a function named emojiIcon:

```
Icon emojiIcon(int scale) {
  switch (scale) {
    case 1:
      return Icon(
        Icons.sentiment_very_dissatisfied,
        size: 48,
        color: Colors.red,
      );
    case 2:
      return Icon(
        Icons.sentiment_dissatisfied,
        size: 48,
        color: Colors.orange,
      );
    case 3:
      return Icon(
        Icons.sentiment_neutral,
        size: 48,
        color: Colors.blue,
      );
    case 4:
      return Icon(
        Icons.sentiment_satisfied,
        size: 48,
        color: Colors.deepPurpleAccent,
      );
    default:
      return Icon(
        Icons.sentiment_very_satisfied,
        size: 48,
        color: Colors.green,
      );
  }
}
```

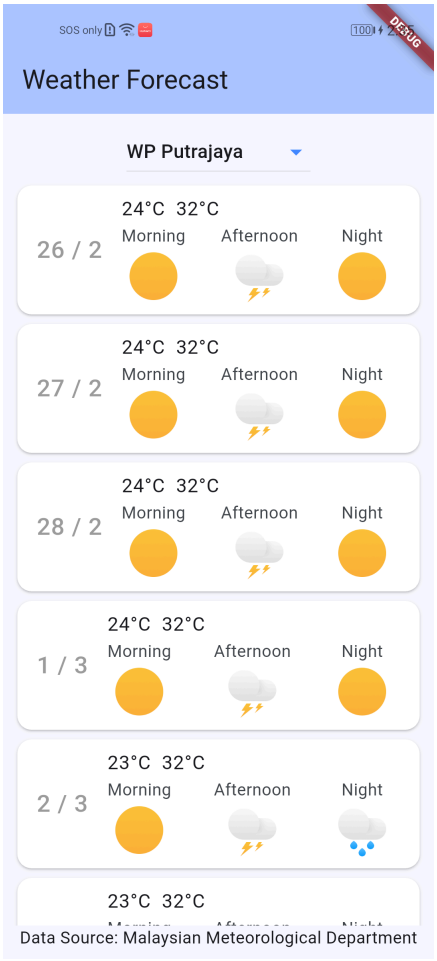
10. Run the program and insert a few emotion journals.

11. Click the menu View -> Tools Windows -> App Inspection. Select your mobile device and your application package name from the list. You will be able to inspect the database created by this app as below:



TODO: Enhance the app by showing the selected emoji.

Practical 10: Weather Web API



This project focuses on building a weather application utilizing data sourced from the Malaysian Meteorological Department (MetMalaysia). The application will primarily display weather-related information. Data will be fetched via API calls, requiring parsing of JSON.

1. Create a new **Application** Flutter project named **weather**.

2. Add the **http** and **intl** packages as a dependency using the **Terminal**.

3. In the main.dart file, import the http package.

```
import 'package:http/http.dart' as http;
```

4. For Android deployment, insert the following to the **Manifest file**.

```
<uses-permission android:name="android.permission.INTERNET" />
```

5. The URL for accessing the weather Open Data API is https://developer.data.gov.my/realtime-api/weather

   Click the link above to find out more about the API.

6. Making call to the API, which retrieve 7-day weather for all the states in Malaysia:
   https://api.data.gov.my/weather/forecast?contains=St@location__location_id

7. The data format retrieved from the web server will be in JSON format as follows: -

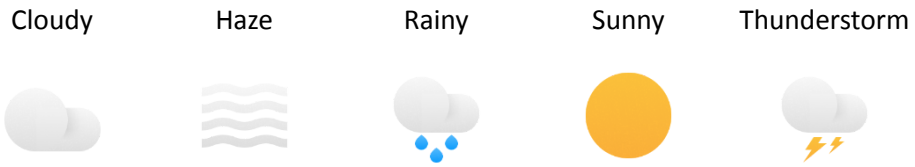
```
[
    {
        "location": {
            "location_id": "St001",
            "location_name": "Perlis"
        },
        "date": "2025-02-16",
        "morning_forecast": "Tiada hujan",
        "afternoon_forecast": "Tiada hujan",
        "night_forecast": "Tiada hujan",
        "summary_forecast": "Tiada hujan",
        "summary_when": "Sepanjang Hari",
        "min_temp": 24,
        "max_temp": 33
    },
    {
        "location": {
            "location_id": "St001",
            "location_name": "Perlis"
        },
        "date": "2025-02-15",
        "morning_forecast": "Tiada hujan",
        "afternoon_forecast": "Tiada hujan",
    . . .
```

8. Create a new dart file named **forecast.dart**. This class will be used to hold weather records obtained from the API call.

```dart
class Forecast {
  final String date;
  final String morning_forecast;
  final String afternoon_forecast;
  final String night_forecast;
  final String summary_forecast;
  final String summary_when;
  final int min_temp;
  final int max_temp;

  const Forecast({
    required this.date,
    required this.morning_forecast,
    required this.afternoon_forecast,
    required this.night_forecast,
    required this.summary_forecast,
    required this.summary_when,
    required this.min_temp,
    required this.max_temp
  });
```

```
  factory Forecast.fromJson(Map<String, dynamic> json) {
    return switch (json) {
      {
      'date': String date,
      'morning_forecast': String morning_forecast,
      'afternoon_forecast': String afternoon_forecast,
      'night_forecast': String night_forecast,
      'summary_forecast': String summary_forecast,
      'summary_when': String summary_when,
      'min_temp': int min_temp,
      'max_temp': int max_temp
      } =>
          Forecast(
              date: date,
              morning_forecast: morning_forecast,
              afternoon_forecast: afternoon_forecast,
              night_forecast: night_forecast,
              summary_forecast: summary_forecast,
              summary_when: summary_when,
              min_temp: min_temp,
              max_temp: max_temp
          ),
      _ => throw const FormatException('Failed to load forecast.'),
    };
  }
}
```

9. Insert the following images to the **assets/images** folder.



10. Open the **pubspec.yaml** file. Then scroll to the **flutter:** section. You are to insert a sub-section named **assets:**

```
53  flutter:
54
55    # The following line ensures that the Material Icons font is
56    # included with your application, so that you can use the icons in
57    # the material Icons class.
58    uses-material-design: true
59
60    #TODO 1 : Insert assets
61    assets:
62      - assets/images/
63    # To add assets to your application, add an assets section, like this:
64    # assets:
65    #   - images/a_dot_burr.jpeg
66    #   - images/a_dot_ham.jpeg
```

Below the **assets:** section, insert **– assets/images/**. This ensures the app can bundle all the images in this directory.

11. Open the **main.dart** file. Within the class **_MyHomePageState**, insert the following variables and map:

```
final Map<String, String> states = {
  'St001': 'Perlis',
  'St002': 'Kedah',
  'St003': 'Pulau Pinang',
  'St004': 'Perak',
  'St005': 'Kelantan',
  'St006': 'Terengganu',
  'St007': 'Pahang',
  'St008': 'Selangor',
  'St009': 'WP Kuala Lumpur',
  'St010': 'WP Putrajaya',
  'St011': 'Negeri Sembilan',
  'St012': 'Melaka',
  'St013': 'Johor',
  'St501': 'Sarawak',
  'St502': 'Sabah',
  'St503': 'WP Labuan',
};

String? _selectedState; // Store the selected state
Future<List<Forecast>>? forecastData; // Store the Future
```

Each state has a unique code with a prefix 'St'. We declare two variables named **_selectedState** and **forecastData** to hold state code selected by the user and weather forecast data.

12. Within the same class, write a function to perform network request:

```
Future<List<Forecast>> _fetchForecastData(String locationId) async {
  if (locationId.isEmpty) {
    //Handle empty location ID
    return Future.value([]);
  }

  final url =
Uri.parse('https://api.data.gov.my/weather/forecast?contains=$locationId@location__loca
tion_id&sort=date');

  try {
    final response = await http.get(url);

    if (response.statusCode == 200) {
      // If the server did return a 200 OK response, then parse the JSON.
      final jsonData = jsonDecode(response.body) as List;
      return jsonData.map((json) => Forecast.fromJson(json)).toList();
    } else {
      // If the server did not return a 200 OK response, then throw an exception.
      throw Exception('Failed to load forecast : ${response.statusCode}');
    }
  } catch (e) {
    throw Exception('Error fetching forecast data : $e');
```

```
  }
}
```

This function will receive a state code (**locationId**) as an input parameter.

13. Within the **build** function of the class **_MyHomePageState,** wrap the Column widget with a Padding widget as follows:

```
body: Center(
  child: Padding(
    padding: const EdgeInsets.all(8.0),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.start,
      children: <Widget>[
         //TODO: Insert a DropdownButton widget

        DropdownButton<String>(
          hint: Text('Select a State'),
          value: _selectedState, // The currently selected value
          iconEnabledColor: Colors.blueAccent,
          borderRadius: BorderRadius.all(Radius.circular(10.0)),
          elevation: 8,
          items: states.entries.map((entry) {
            return DropdownMenuItem<String>(
              value: entry.key, // Value when selected
              child: Text(entry.value), // Text displayed in the dropdown
            );
          }).toList(),
          onChanged: (String? newValue) {
            if (newValue != null) {
              setState(() {
                _selectedState = newValue; // State code
                forecastData = _fetchForecastData(newValue);
              });
            }
          },
        ),
         //TODO: Insert an Expanded widget
        Expanded(
          child: FutureBuilder<List<Forecast>>(
              future: forecastData,
              builder: (context, snapshot) {
                if (snapshot.connectionState == ConnectionState.waiting &&
                    forecastData != null) {
                  return CircularProgressIndicator(
                    strokeAlign: BorderSide.strokeAlignCenter,
                  );
                } else if (snapshot.hasError) {
                  return Text('Error: ${snapshot.error}');
                } else if (snapshot.hasData &&
                    snapshot.data!.isNotEmpty) {
                  return ForecastList(forecastData: snapshot.data!);
                } else {
                  return Text('');
                }
              }),
        ),
         //TODO: Insert a Text widget
        Text('Data Source: Malaysian Meteorological Department'),
      ],
    ),
```

The **ForecastList** is a custom widget that we will insert later.

14. At module level, below the class _MyHomePageState, insert a map for weather status and image file:

```
final Map<String, String> weather_status = {
  'Berjerebu': 'haze.png',
  'Tiada hujan': 'sunny.png',
  'Hujan': 'rainy.png',
  'Hujan di beberapa tempat': 'rainy.png',
  'Hujan di satu dua tempat': 'rainy.png',
  'Hujan di satu dua tempat di kawasan pantai': 'rainy.png',
  'Hujan di satu dua tempat di kawasan pedalaman': 'rainy.png',
  'Ribut petir': 'thunderstorm.png',
  'Ribut petir di beberapa tempat': 'thunderstorm.png',
  'Ribut petir di beberapa tempat di kawasan pedalaman  Scattered':
  'thunderstorm.png',
  'Ribut petir di satu dua tempat': 'thunderstorm.png',
  'Ribut petir di satu dua tempat di kawasan pantai': 'thunderstorm.png',
  'Ribut petir di satu dua tempat di kawasan pedalaman': 'thunderstorm.png',
};
```

15. Next, insert a new stateless widget named **ForecastList**:

```
class ForecastList extends StatelessWidget {
  const ForecastList({super.key});

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

16. Declare a variable named forecastData as final and make it a required parameter for the default constructor:

```
class ForecastList extends StatelessWidget {
  const ForecastList2({super.key, required this.forecastData});

  final List<Forecast> forecastData;

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

17. Within the **build** function, replace the **Placeholder** with a **ListView.builder**.

```
@override
Widget build(BuildContext context) {
  return ListView.builder(
      itemCount: forecastData.length,
      itemBuilder: (context, index){});
}
```

18. Within the **itemBuilder** function, write the following code:

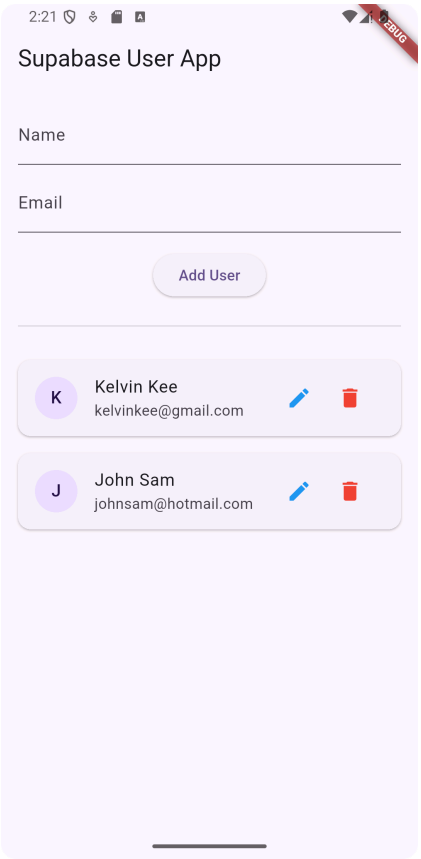
```
  return ListView.builder(
        itemCount: forecastData.length,
        itemBuilder: (context, index) {
          final forecast = forecastData[index];
          return Card(
            color: Colors.white,
            child: ListTile(
              title: Text('${forecast.min_temp}°C  ${forecast.max_temp}°C',),
              leading:
                  Text('${DateFormat('yyyy-MM-dd').parse(forecast.date).day} /
                        ${DateFormat('yyyy-MM-dd').parse(forecast.date).month}',
                      style: TextStyle(color: Colors.grey, fontSize: 20,)),
              subtitle: Row(
                crossAxisAlignment: CrossAxisAlignment.center,
                children: [
                  Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      Text('Morning'),
                      weather_status[forecast.morning_forecast] != null
                          ? Image.asset(
                       'assets/images/${weather_status[forecast.morning_forecast].toString()}',
                                width: 48, height: 48,)
                          : Image.asset('assets/images/unknown.png', width: 48, height: 48,),
                    ],
                  ),
                  Expanded(child: SizedBox(),),
                  Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      Text('Afternoon'),
                      weather_status[forecast.afternoon_forecast] != null
                      ? Image.asset(
                     'assets/images/${weather_status[forecast.afternoon_forecast].toString()}',
                             width: 48, height: 48,)
                          : Image.asset('assets/images/unknown.png', width: 48, height: 48,),
                    ],
                  ),
                  Expanded(child: SizedBox(),),
                  Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      Text('Night'),
                      weather_status[forecast.night_forecast] != null
                          ? Image.asset(
                       'assets/images/${weather_status[forecast.night_forecast].toString()}',
                                width: 48, height: 48,)
                          : Image.asset('assets/images/unknown.png', width: 48, height: 48,
                            ),
                    ],
                  ),
                ],
              ),
            ),
          );
        });
  }
}
```

19. Run the program and select a state from the given list to see 7-day weather forecast records.

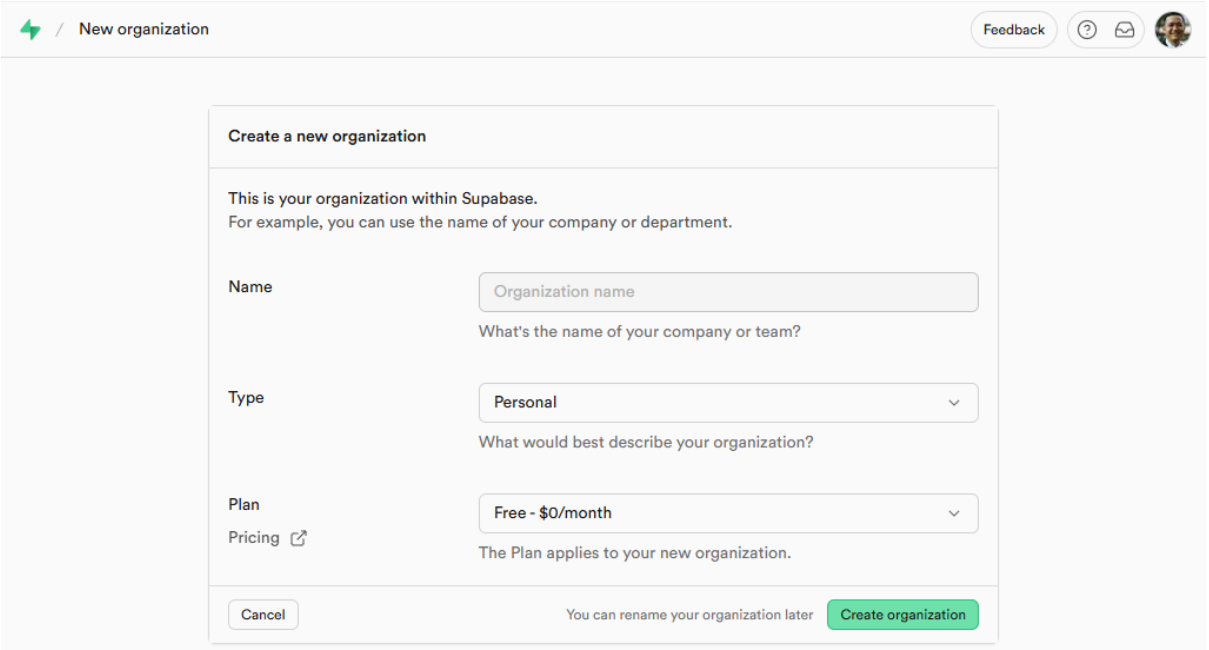
TODO: Enhance the app by saving the selected state name using the Shared Preference.

Practical 11: Supabase – Backend as a Service (BaaS)

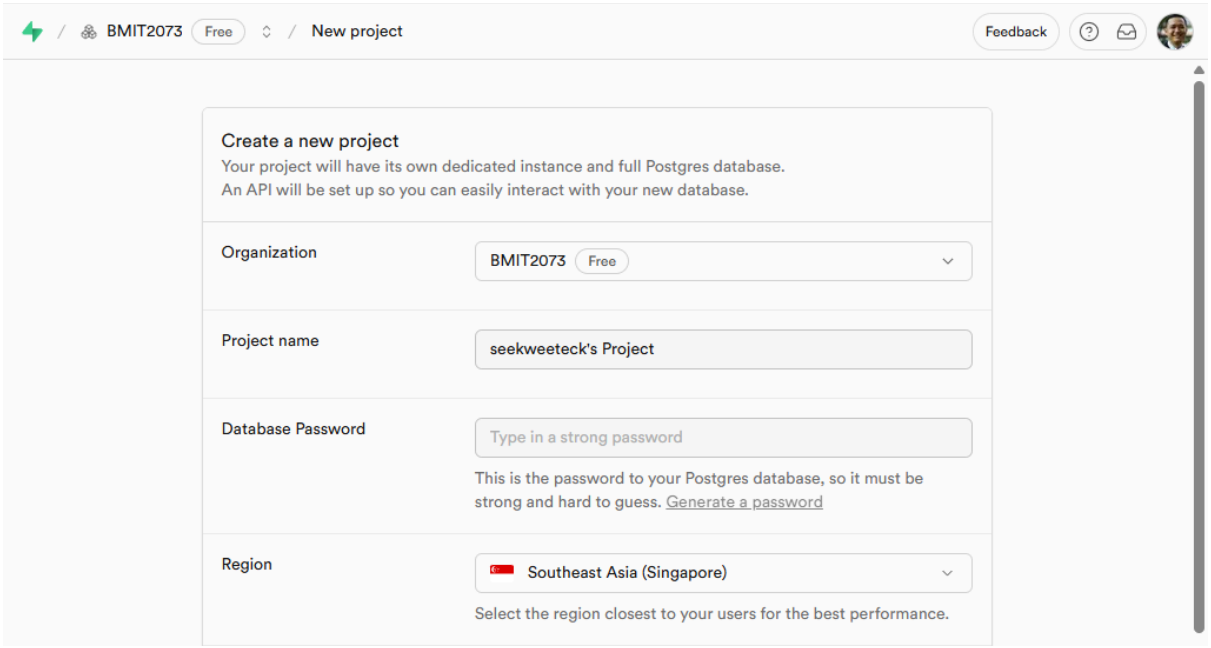
Supabase, an alternative to Firebase, provides a full Postgres database for every project with Realtime functionality, database backups, extensions, and more. This lab demonstrates the processes of creating a Flutter app that can perform CRUD of contact records as shown below:



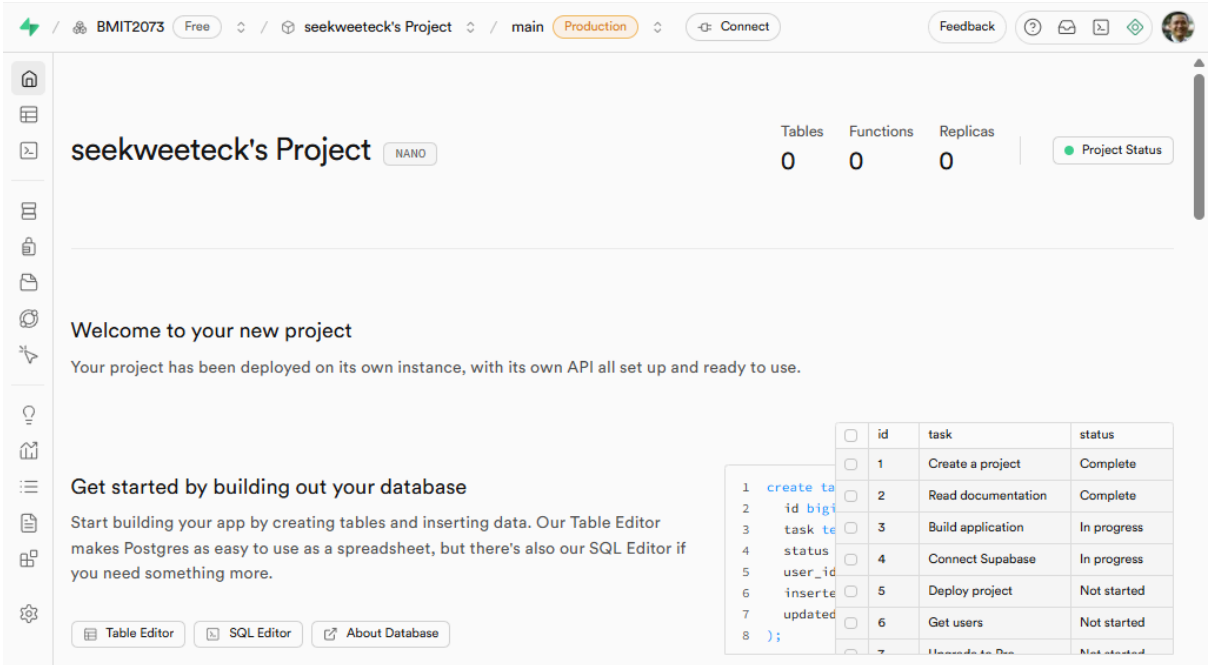
1. Visit https://supabase.com/, register a new organisation. Select a type and opt for the **Free** plan. Press the **Create organization** button to proceed.



2. Next, assign a **project name** and select a **region,** e.g. Southeast Asia (Singapore). Press the **Create project** button to continue.

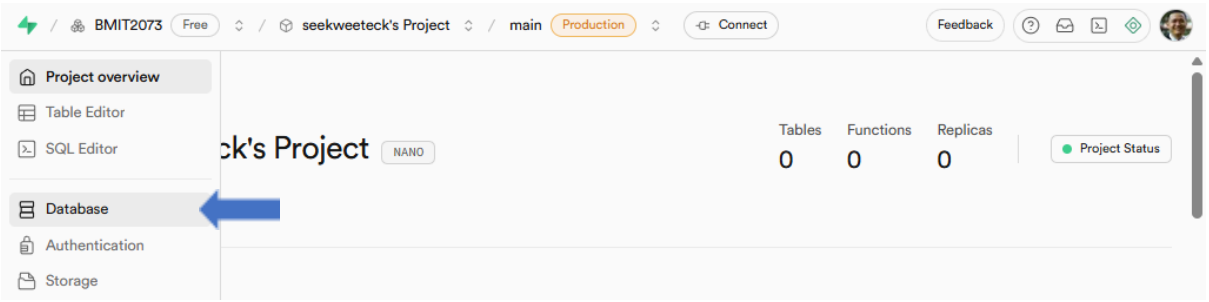


3. Once your project has been created, you will be prompted to the dashboard of your project. Scroll to the bottom to view main features offered by Supabase.

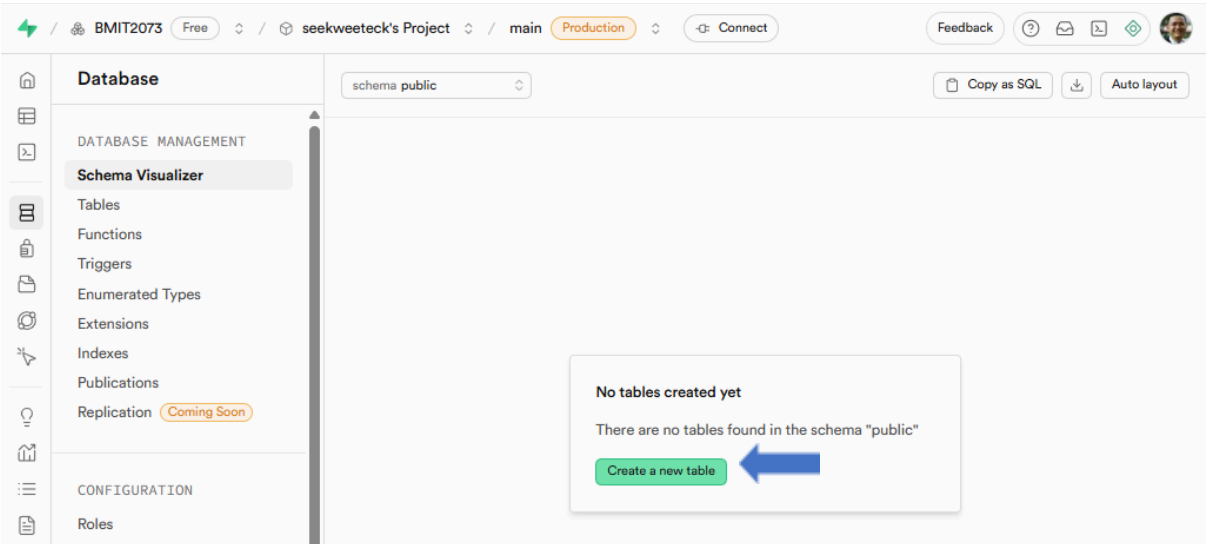


Some of the core features are PostgreSQL database, authentication, Realtime engine, storage, etc. Visit https://supabase.com/docs/guides/getting-started/features to find out more.

4. Next, let us create a new database. Click on the left panel and select **Database**.

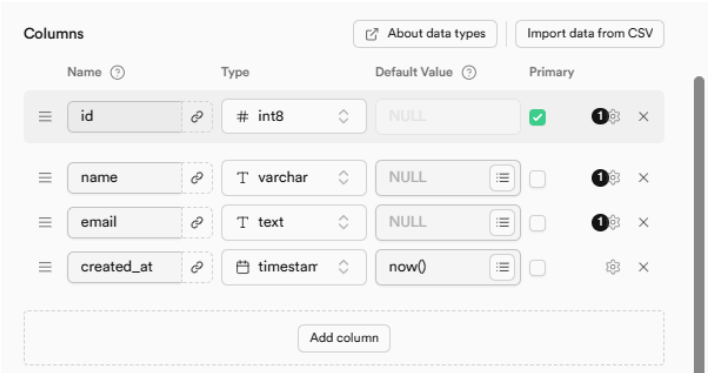


5. Click the **Create a new table** button to continue.

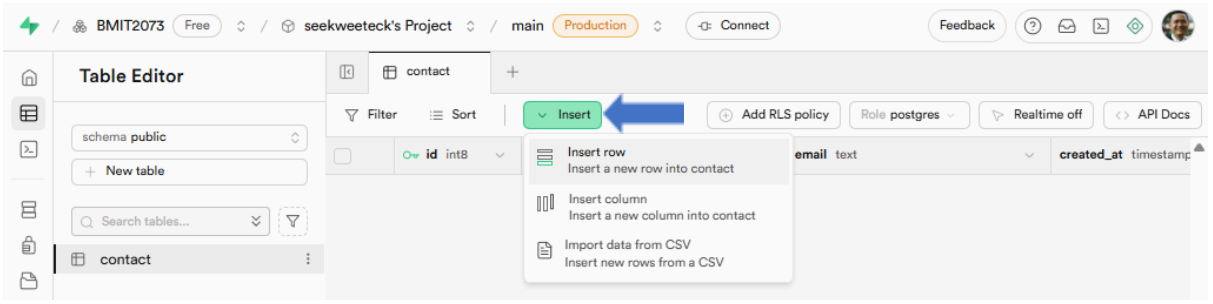


6. Enter a table **name** and scroll down to the bottom.

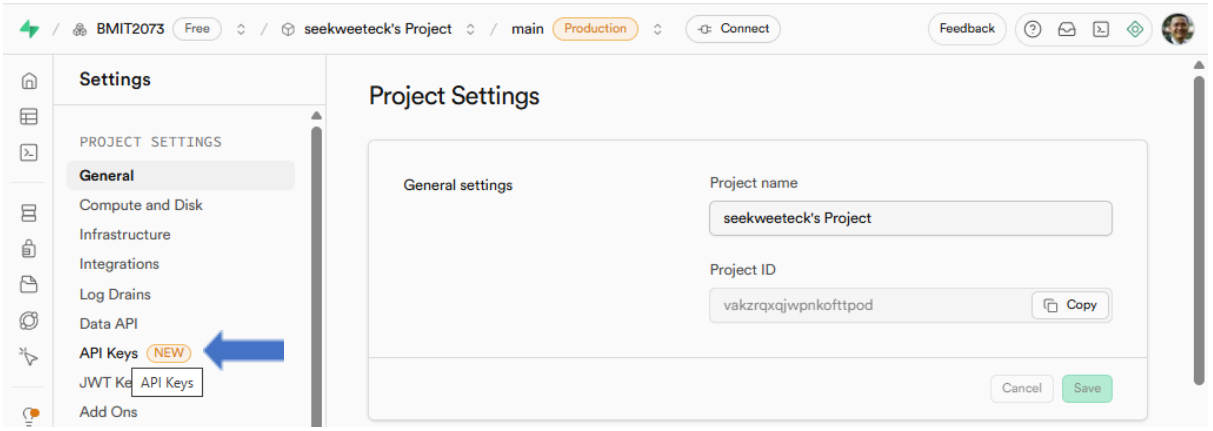
7. Add two additional columns; **name** and **email**. Click the **Save** button to complete the task.



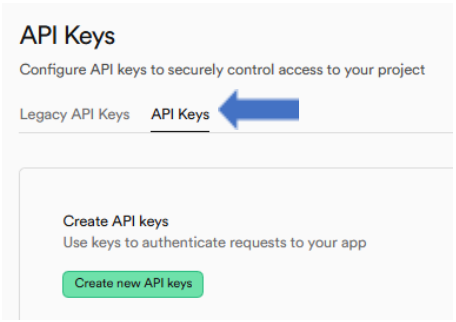
8. On the top panel, click the **Insert -> Insert row** to insert some records into the **contact** table.



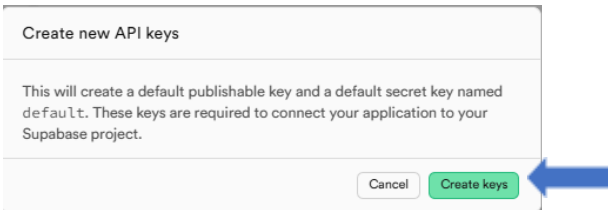
9. Next, we will enable the backend API key, which will allow applications to access the database. On the left panel, click **Project Settings** (Project Settings) and then click **API Keys**.



10. Next, click the API **Keys** tab and then the **Create new API keys** button.

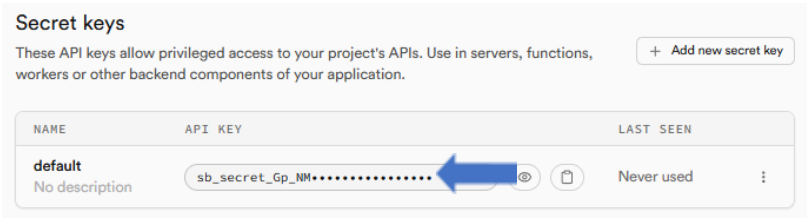


Click **Create keys** when prompt.

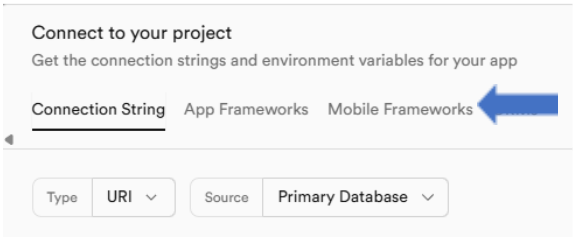


11. Next, click the API **Keys** tab and then the **Create new API keys** button.

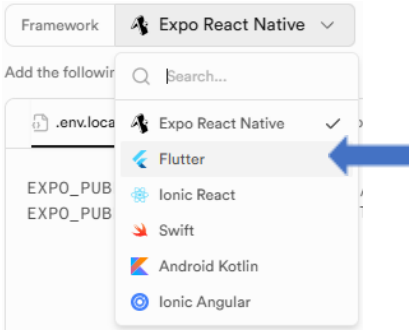
12. Copy and paste the **Secret keys** to a text file. You will need it later in your Flutter app.



13. On the top panel of the dashboard, click the **Connect** ( ) button, and then click the **Mobile Frameworks** tab.



14. In the Framework dropdown list, select **Flutter**.



15. Copy and paste the **server URL** to a text file.

16. Next, let us create a new Flutter mobile app. Start **Android Studio**, and create a new Flutter project using the **Empty** template.

17. Once your project is ready, open a new terminal window, enter the following command to insert the Supabase dependencies:

```
flutter pub add supabase_flutter
```

18. Enter the following codes. Also, copy and paste the **server URL** and **Secret keys** to the program code**.**

```
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

const String supabaseUrl = 'SERVER URL';
const String supabaseKey = 'API KEY';

class User {
  final String id;
  final String name;
  final String email;

  User({required this.id, required this.name, required this.email});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['id'].toString(), // Supabase 'id' field is an integer/bigint
      name: json['name'] as String,
      email: json['email'] as String,
    );
  }
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Supabase.initialize(
    url: supabaseUrl,
    anonKey: supabaseKey,
  );
  runApp(MyApp());
}

final supabase = Supabase.instance.client;

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Supabase',
      theme: ThemeData(

        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
      ),
      home: const UserListPage(),
    );
  }
}

class UserListPage extends StatefulWidget {
  const UserListPage({super.key});

  @override
  State<UserListPage> createState() => _UserListPageState();
}
```

```dart
class _UserListPageState extends State<UserListPage> {
  // Supabase client instance
  final supabase = Supabase.instance.client;

  // List to hold the fetched users
  List<User> _users = [];
  bool _isLoading = true;

  // Text controllers for input fields
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();

  // A selected user for updating or deleting
  User? _selectedUser;

  @override
  void initState() {
    super.initState();
    _fetchUsers();
  }

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    super.dispose();
  }

  // Fetches all users from the 'user' table
  Future<void> _fetchUsers() async {
    setState(() {
      _isLoading = true;
    });

    try {
      // Use select() to get all rows from the 'user' table.
      final response = await supabase.from('contact').select();

      // Parse the list of JSON objects into a list of User objects.
      final users = (response as List).map((item) => User.fromJson(item)).toList();

      setState(() {
        _users = users;
      });
    } catch (e) {
      // Handle any errors during the fetch
      print('Error fetching users: $e');
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Failed to fetch users: $e')),
      );
    } finally {
      setState(() {
        _isLoading = false;
      });
    }
  }

  // Adds a new user to the 'user' table
  Future<void> _addUser() async {
    final name = _nameController.text.trim();
    final email = _emailController.text.trim();
```

```
  if (name.isEmpty || email.isEmpty) {
    return;
  }

  try {
    setState(() {
      _isLoading = true;
    });
    // Use insert() to add a new row to the 'user' table.
    final response = await supabase.from('contact').insert({
      'name': name,
      'email': email,
    }).select();

    _nameController.clear();
    _emailController.clear();

    // Since we used .select(), Supabase returns the inserted row.
    // We can add it directly to our list to avoid re-fetching all data.
    final newUser = User.fromJson((response as List).first);
    setState(() {
      _users.add(newUser);
    });

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('User added successfully!')),
    );
  } catch (e) {
    print('Error adding user: $e');
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Failed to add user: $e')),
    );
  } finally {
    setState(() {
      _isLoading = false;
    });
  }
}

// Updates an existing user in the 'user' table
Future<void> _updateUser() async {
  if (_selectedUser == null) {
    return;
  }

  final name = _nameController.text.trim();
  final email = _emailController.text.trim();

  if (name.isEmpty && email.isEmpty) {
    return;
  }

  try {
    setState(() {
      _isLoading = true;
    });
    // Use update() and eq() to target a specific row by its 'id'.
    final response = await supabase.from('contact').update({
      'name': name.isNotEmpty ? name : _selectedUser!.name,
      'email': email.isNotEmpty ? email : _selectedUser!.email,
```

```
      }).eq('id', _selectedUser!.id).select();

      _nameController.clear();
      _emailController.clear();

      // Find and update the user in our local list.
      final updatedUser = User.fromJson((response as List).first);
      final index = _users.indexWhere((u) => u.id == updatedUser.id);
      if (index != -1) {
        setState(() {
          _users[index] = updatedUser;
        });
      }

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('User updated successfully!')),
      );
    } catch (e) {
      print('Error updating user: $e');
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Failed to update user: $e')),
      );
    } finally {
      setState(() {
        _isLoading = false;
        _selectedUser = null;
      });
    }
  }

  // Deletes a user from the 'user' table
  Future<void> _deleteUser(String userId) async {
    try {
      setState(() {
        _isLoading = true;
      });
      // Use delete() and eq() to remove a specific row.
      await supabase.from('contact').delete().eq('id', userId);

      // Remove the user from our local list to update the UI.
      setState(() {
        _users.removeWhere((user) => user.id == userId);
      });

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('User deleted successfully!')),
      );
    } catch (e) {
      print('Error deleting user: $e');
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Failed to delete user: $e')),
      );
    } finally {
      setState(() {
        _isLoading = false;
      });
    }
  }

  // Fills the text fields with the selected user's data for editing
  void _selectUserForUpdate(User user) {
```

```
    setState(() {
      _selectedUser = user;
      _nameController.text = user.name;
      _emailController.text = user.email;
    });
  }

  // Clears the text fields and selection
  void _clearSelection() {
    setState(() {
      _selectedUser = null;
      _nameController.clear();
      _emailController.clear();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Supabase User App'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            // Input fields for name and email
            TextField(
              controller: _nameController,
              decoration: const InputDecoration(labelText: 'Name'),
            ),
            const SizedBox(height: 8),
            TextField(
              controller: _emailController,
              decoration: const InputDecoration(labelText: 'Email'),
            ),
            const SizedBox(height: 16),
            // Action buttons
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceEvenly,
              children: [
                ElevatedButton(
                  onPressed: _selectedUser != null ? _updateUser : _addUser,
                  child: Text(_selectedUser != null ? 'Update User' : 'Add User'),
                ),
                if (_selectedUser != null)
                  TextButton(
                    onPressed: _clearSelection,
                    child: const Text('Cancel'),
                  ),
              ],
            ),
            const SizedBox(height: 16),
            const Divider(),
            const SizedBox(height: 16),
            // User list view
            Expanded(
              child: _isLoading
                  ? const Center(child: CircularProgressIndicator())
                  : _users.isEmpty
                  ? const Center(child: Text('No users found. Add one!'))
```

```
                  : ListView.builder(
                itemCount: _users.length,
                itemBuilder: (context, index) {
                  final user = _users[index];
                  return Card(
                    margin: const EdgeInsets.symmetric(vertical: 8),
                    child: ListTile(
                      leading: CircleAvatar(child: Text(user.name[0])),
                      title: Text(user.name),
                      subtitle: Text(user.email),
                      trailing: Row(
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          IconButton(
                            icon: const Icon(Icons.edit, color: Colors.blue),
                            onPressed: () => _selectUserForUpdate(user),
                          ),
                          IconButton(
                            icon: const Icon(Icons.delete, color: Colors.red),
                            onPressed: () => _deleteUser(user.id),
                          ),
                        ],
                      ),
                    ),
                  );
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

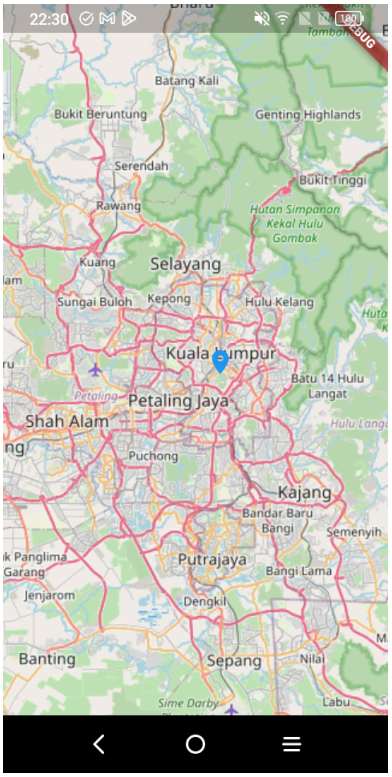
That's all for this practical!

TODO: Visit https://supabase.com/docs/guides/getting-started/tutorials/with-flutter to learn more about Supabase.

Find out how Supabase can do the followings: -

- Utilise advanced database features such as stored procedure, trigger, functions etc.
- Custom Python Backend API

Practical 12: Open Street Map



OpenStreetMap (OSM) is a free, editable map of the world that's created and maintained by volunteers. It's a collaborative project that uses open data to create detailed maps of geographic locations, landmarks, and more. This lab will create a simple map utilizing OSM with a marker.

1. Create a new Flutter app using the **Empty** template named **my_osm**.
2. Open the Terminal and get the following dependencies:

```
flutter pub add flutter_map
flutter pub add  flutter_map_location_marker
```

3. In the **main.dart** file, import the following:

```
import 'package:flutter_map/flutter_map.dart';
import 'package:latlong2/latlong.dart';
```

4. Replace the **Text** widget with a FlutterMap**.**

```
child: FlutterMap(
  options: const MapOptions(
    initialZoom: 10,
    //TODO: Insert latitude and longitude
    initialCenter: const LatLng(3.140853, 101.693207),
  ),
  children: [
    TileLayer(
      maxZoom: 19,
      urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
      userAgentPackageName: 'com.example.demo_openscreetmap',
    ),



    const MarkerLayer(
```

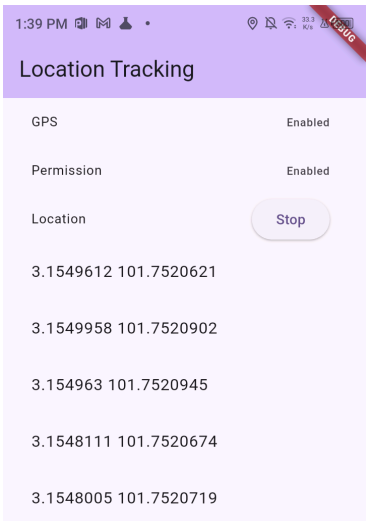
```
          markers: [
            Marker(
                point: LatLng(3.140853, 101.693207),
                child: Icon(
                  Icons.location_on,
                  color: Colors.blue,
                ))
          ],
        )
      ],
    ),
```

5. Save and run your app.

TODO: Visit https://wiki.openstreetmap.org/wiki/How_to_contribute and learn to contribute map data.

Practical 13: Location Tracking

1. We will create an app using location tracking using device GPS or wireless network.



2. Create a new mobile app using the Application template.

3. Insert the following dependencies:
   location
   permission_handler

4. Open the android/app/src/main/AndroidManifest.xml file. Insert the following permissions.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- TODO 1 - Insert android permissions -->
 <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
 <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

 <application ...>
```

   The ACCESS_FINE_LOCATION includes GPS data in reporting user location.
   The ACCESS_COARSE_LOCATION includes data from the most battery-efficient non-GPS provider available (for example, the network).

5. Open the main.dart file. Import the following libraries:

```
//TODO 2 - Import libraries
import 'dart:async';
import 'dart:developer';
import 'package:permission_handler/permission_handler.dart' as handler;
import 'package:location/location.dart';
```

6. Within the _MyHomePageState, insert the following:

```
//TODO 3 - Declare local variables
bool _permissionGranted = false;
bool _gpsEnabled = false;
bool _trackingEnabled = false;
late StreamSubscription _subscription;
final Location _location = Location();
final List<LocationData> _locationList = [];
```

7. Within the _MyHomePageState, create two async methods to check location and GPS permission status:

```
//TODO 4 - Check is permission granted
Future<bool> isPermissionGranted() async {
  return await handler.Permission.locationWhenInUse.isGranted;
}

//TODO 5 - Check is GPS enabled
Future<bool> isGpsEnabled() async {
  return await handler.Permission.location.serviceStatus.isEnabled;
}
```

8. Insert the checkStatus method, which determines the location and GPS permission status.

```
//TODO 6 - Check permission status
  void checkStatus() async {
    bool permissionGranted = await isPermissionGranted();
    bool gpsEnabled = await isGpsEnabled();
    setState(() {
      _permissionGranted = permissionGranted;
      _gpsEnabled = gpsEnabled;
    });
  }
```

9. Override the initState() method.

```
//TODO 7 - check status on initialisation
  @override
  void initState() {
    super.initState();
    checkStatus();
  }
```

10. Within the _MyHomePageState, create to async methods that start or end location tracking:

```
 //TODO 8 - start tracking user's location
  void startTracking() async {
    if (!(await isGpsEnabled())) {
      return;
    }
    if (!(await isPermissionGranted())) {
      return;
    }
    _subscription = _location.onLocationChanged.listen((event) {
      addLocation(event);
    });
    setState(() {
      _trackingEnabled = true;
    });
  }
```

```
  //TODO 9 - stop tracking user's location
  void stopTracking() {
    _subscription.cancel();
    setState(() {
      _trackingEnabled = false;
    });
    clearLocation();
  }
```

11. Override the dispose method. We should stop the location tracking when the app enters the dispose stage.

```
  //TODO 10 - stop tracking on dispose
  @override
  void dispose() {
    stopTracking();
    super.dispose();
  }
```

12. Within the _MyHomePageState, we may request user permission to enable GPS location service:

```
//TODO 11 - Request for GPS
  void requestEnableGps() async {
    if (_gpsEnabled) {
      log("Already open");
    } else {
      bool isGpsActive = await _location.requestService();
      if (!isGpsActive) {
        setState(() {
          _gpsEnabled = false;
        });
        log("User did not turn on GPS");
      } else {
        log("gave permission to the user and opened it");
        setState(() {
          _gpsEnabled = true;
        });
      }
    }
  }
```

13. Similarly, we may also request permission to access user's location:

```
//TODO 12 - Request location permission
  void requestLocationPermission() async {
    var permissionStatus =
    await handler.Permission.locationWhenInUse.request();

    if (permissionStatus == PermissionStatus.granted) {
      setState(() {
        _permissionGranted = true;
      });
    } else {
      setState(() {
        _permissionGranted = false;
      });
    }
  }
```

14. From time to time, location data delivered by location service are inserted to a location list:

```
//TODO 13 - Add a new location record to the location list
void addLocation(LocationData data) {
  setState(() {
    _locationList.insert(0, data);
  });
}
```

15. When user press the Stop button, we shall clear location data from the location list:

```
//TODO 14 - Clear the location list
void clearLocation() {
  setState(() {
    _locationList.clear();
  });
}
```

16. Create a widget to display a location record:

```
//TODO 15 - Create a list tile template
ListTile buildListTile(
    String title,
    Widget? trailing,
    ) {
  return ListTile(
    dense: true,
    title: Text(title),
    trailing: trailing,
  );
}
```

17. Within the build method, create the main UI:

```
//TODO 16 - Create the UI
      body:Padding(
        padding: const EdgeInsets.symmetric(horizontal: 12),
        child: Column(
          mainAxisSize: MainAxisSize.max,
          mainAxisAlignment: MainAxisAlignment.start,
          children: [
            buildListTile(
              "GPS",
              _gpsEnabled
                  ? const Text("Enabled")
                  : ElevatedButton(
                  onPressed: () {
                    requestEnableGps();
                  },
                  child: const Text("Enable Gps")),
            ),
```

```
        buildListTile(
          "Permission",
          _permissionGranted
              ? const Text("Enabled")
              : ElevatedButton(
              onPressed: () {
                requestLocationPermission();
              },
              child: const Text("Request Permission")),
        ),
        buildListTile(
          "Location",
          _trackingEnabled
              ? ElevatedButton(
              onPressed: () {
                stopTracking();
              },
              child: const Text("Stop"))
              : ElevatedButton(
              onPressed: _gpsEnabled && _permissionGranted
                  ? () {
                startTracking();
              }
                  : null,
              child: const Text("Start")),
        ),
        Expanded(
            child: ListView.builder(
              itemCount: _locationList.length,
              itemBuilder: (context, index) {
                return ListTile(
                  title: Text(
                      "${_locationList[index].latitude}
                      ${_locationList[index].longitude}"),
                );
              },
            ))
      ],
    ),
  ),
```

TODO: Enhance the app by showing the user current location using the Open Street Map.